HW5: Classification and Representation (due 1/9 23:30)

莊明儒 111024513 / 王子淇 111024523

Contents

0.1 Problem 1: Fault Classification 1
0.2 Problem 2: Forecast for SP500 returns 24
0.3 Task 1 : Regression 28
0.4 2.1.b.1 SVR - Radial kernel 31
0.5 2.1.b.2 SVR - Linear kernel 33
0.6 Task 2 : Classification 35
0.7 2.2.b.1 SVM - Radial kernel 38
0.8 2.2.b.2 SVM - Linear kernel 41

1 2.2.c Logistics regression 44

1.1 2.3 Summary 46

0.1 Problem 1: Fault Classification

This problem concerns about the quality of a casting item. This data set collects the images (100x100 pixels) of 6633 items, together with their quality labels (defective or ok).

The objective is to predict the label (defect or ok) of each image.

0.1.1 Your analysis should include the following:

- The training procedures of the classifiers (at least 2 different classifiers), and their performance comparisons.
- A brief analysis summary.
- (Bonus) Can defect items (image data) be clustered with patterns (sub-types)? Explore your findings.

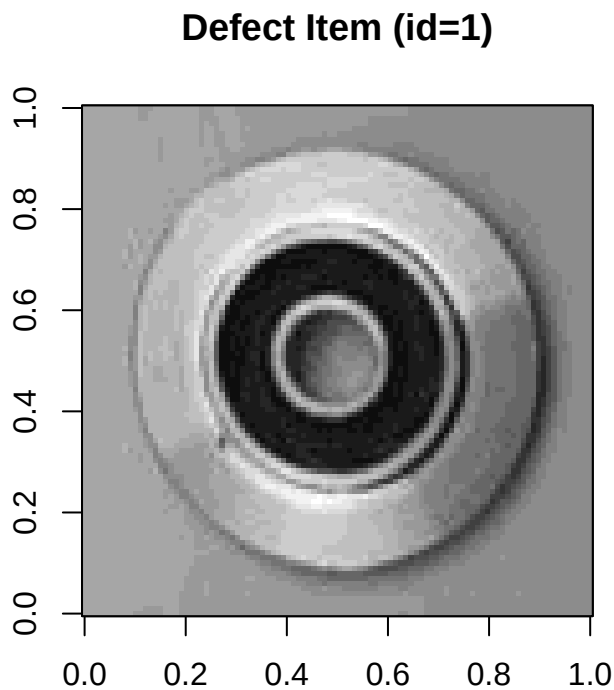
0.1.2 Read image into data frame

```
#install required packages
library("jpeg")
library("plyr")
```

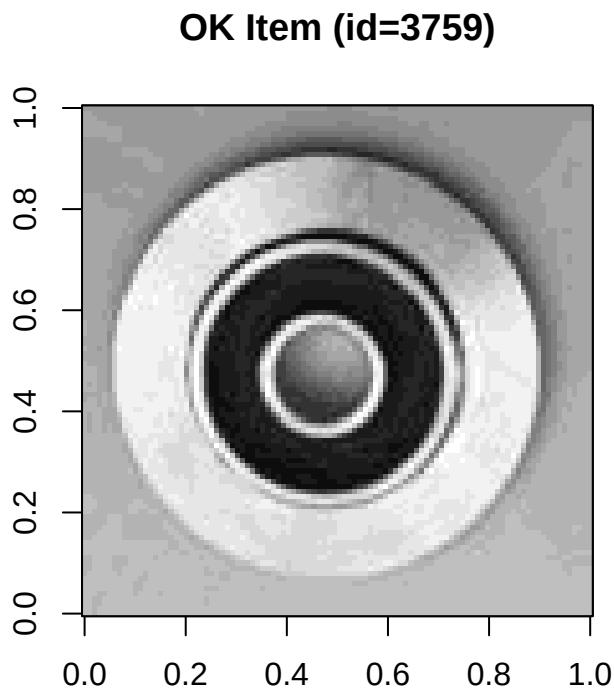
```r
setwd("casting_100x100")

def_paths <- dir("def_front")
ok_paths <- dir("ok_front")

data_def = sapply(1:length(def_paths), function(x){
  tmp = readJPEG(paste0("def_front/", def_paths[x]))[,,1]
  tmp = as.vector(tmp)
  return(tmp)
})

data_ok = sapply(1:length(ok_paths), function(x){
  tmp = readJPEG(paste0("ok_front/", ok_paths[x]))[,,1]
  tmp = as.vector(tmp)
  return(tmp)
})

data = as.data.frame(rbind(t(data_def), t(data_ok)))
data$y = as.factor(c(rep(1,length(def_paths)), rep(0,length(ok_paths))))
dim(data)
```

```
## [1]  4805 10001
```

```r
## Data Preprocessing

show_image = function(img_cast, col = gray(1:20/20), ...){
  image(matrix(as.matrix(img_cast), ncol = 100, byrow = TRUE)[, 100:1], col = col,
        cex.main = 6, ...)
}

## show images for a defect item and a good (OK) item
par(mfrow = c(1,1), pty="s")
show_image(data[1,-10001], col = gray(1:20/20)); box(); title("Defect Item (id=1)")
```




```
show_image(data[length(def_paths)+1,-10001], col = gray(1:20/20)); box(); title("OK Item (id=3759)")
```



Your analysis starts here…

0.1.3 Train-test splitting

In order to evaluate the prediction performance of the classifiers trianed in the following analysis, the data is splitted into training set and testing set. The training set and the testing set respectively accounts for 80% (3844 images) and 20% (961 observations) of the original data. The splitting is determined by simple random sampling without replacement (with random seed = 10).

```
train = rep(FALSE, nrow(data))
set.seed(10)
train[sample(1:nrow(data), round(nrow(data) * 0.8))] = TRUE
test = !train
```

0.1.4 Data reduction

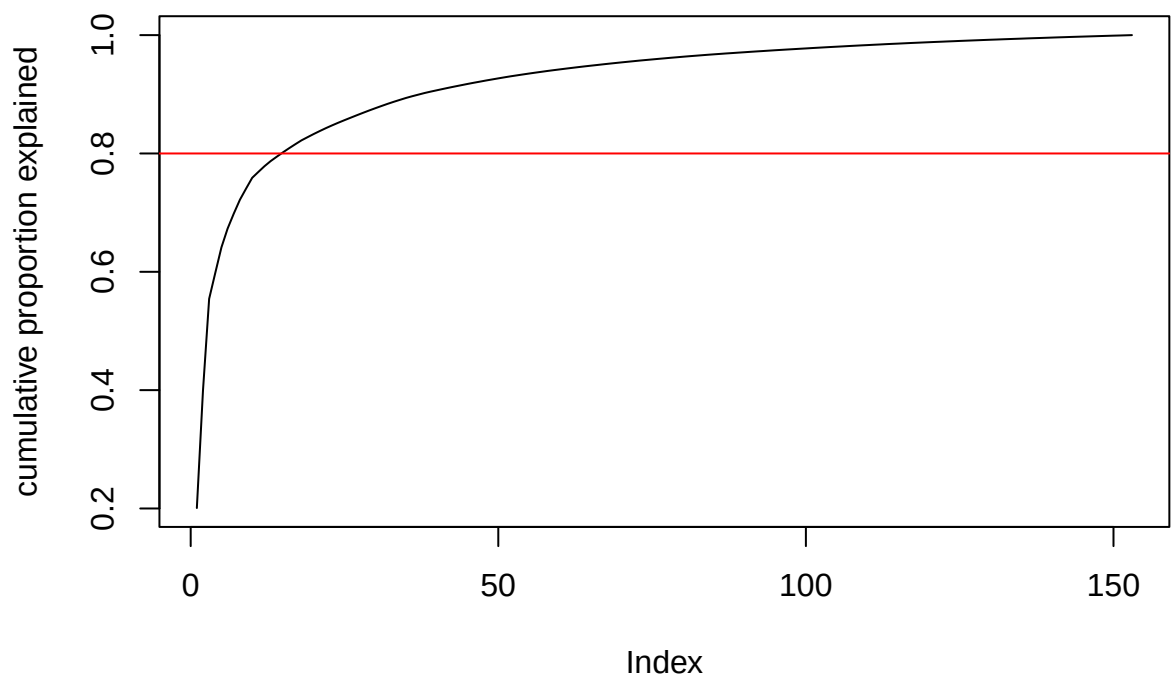
For principal component analysis, we scaled the data. Such measure was taken in hope of having good interpretability of the result of eigenvalue decomposition of the covariance matrix.

```
scale.data = array(NA, dim = dim(data))
# scaling for training set
scale.data[train,] = cbind(scale(data[train, -10001]), data$y[train])
```

```r
# scaling for testing set (use the mean and sd of the training set)
train.m = matrix(rep(apply(data[train, -10001], 2, mean), each = nrow(scale.data)), nrow = nrow(scale.da
train.sd = matrix(rep(apply(data[train, -10001], 2, sd), each = nrow(scale.data)), nrow = nrow(scale.dat
scale.data[test,] = (data[test, -10001])-train.m)/train.sd
write.csv(scale.data, "Q1_scale_data.csv")
```

```r
X = scale.data[train, -10001]
eigen.decomp = eigen(cov(X))
eigen.decomp$values = eigen.decomp$values[1:153]
eigen.decomp$vectors = eigen.decomp$vectors[,1:153]
dump("eigen.decomp", "eigen.decomp.Rdata")
```

```r
V = eigen.decomp$vectors
D = eigen.decomp$values
cum.p = cumsum(D)/sum(D)
plot(cum.p, type = "l", ylab = "cumulative proportion explained")
abline(h = 0.8, col = "red")
```




```r
sum(cum.p<0.8)
```

```
## [1] 14
```

```
pc.train = as.matrix(scale.data[train, -10001]) %*% V[,1:21]
pc.train = as.data.frame(pc.train)
pc.train$y = data[train,]$y
colnames(pc.train)[-22] = paste("PC", 1:21, sep = "")
head(pc.train)
dump("pc.train", "Q1_pc_train.Rdata")
```

0.1.5 The training procedures of the classifiers (at least 2 different classifiers), and their performance comparisons.

In the following, two classifiesrs, random forest and support vector machine (svm) will be trained. The associated cross validation (cv) procedrure for tuning hyperparameters will be shown, as well as the testing results. For the part of cv, five-fold is adopted where the folds are randomly determined as follows.

```
k = 5
set.seed (10)
folds=sample(1:k, nrow(pc.train), replace=TRUE)
folds.n = table(folds)
```

a. Random forest

Throughout the cross-validation procedure, the tuning parameter m is searched among 2 to 21.

```
params = seq(1, 21, 4)
```

```
cv.errors=matrix(NA, k, length(params))
for(i in 1:length(params)){
  param = params[i]
  print(param)
  for(j in 1:k){
      fit = randomForest(y~., data = pc.train, subset = folds != j,
                         split = "deviance", mtry = params[i])
      pred = predict(fit, newdata = pc.train[folds == j,])
      confuse = table(pc.train[folds == j,]$y, pred)
      acc = sum(diag(confuse))/sum(confuse)
      cv.errors[j, i] = acc
  }
}
dump("cv.errors", "Q1_rf1_cv_err.RData")
```

```
cv = apply(cv.errors*matrix(rep(folds.n, length(params)), nrow = k), 2, sum)/nrow(pc.train)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "m", ylab = "cv Accuracy")
opt.m = params[which.max(cv)]
abline(v = params[which.max(cv)], col = "red")
legend("topleft", c("cv Accuracy", "CI"), lty = 1:2)
```

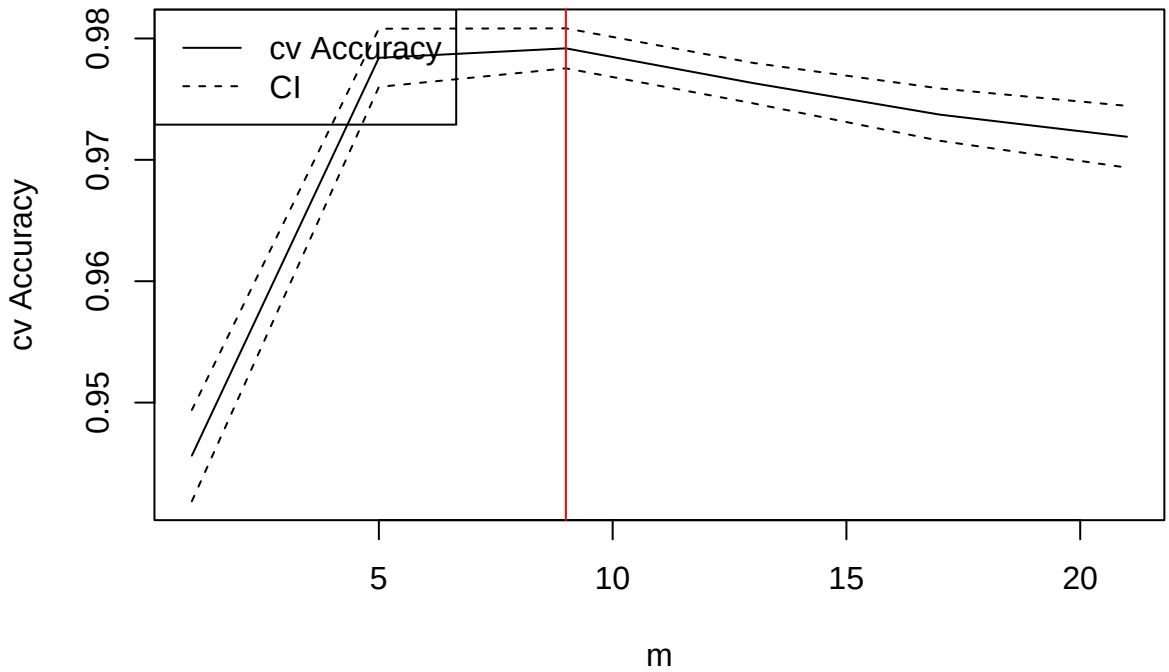


Figure 1: (#fig:rf1_cv)Cross validation for the tuning parameter (m)

Figure @ref(fig:rf1_cv) suggests that it might be better to set m at 9 for the highest accuracy estimate. This result also implies that bagging, which use m equal to the number of all predictors, might not be a good choice. We thus fit again the random forest under m = 9 using the whole training data.

```
rf1 = randomForest(y~., data = pc.train, subset = folds != j,
                           split = "deviance", mtry = opt.m)
dump("rf1", "Q1_rf1.RData")
```

```
plot(rf1, main = "")
legend("topright", c("OOB", "False negative rate", "False positive rate"), col = 1:3, lty = 1)
```

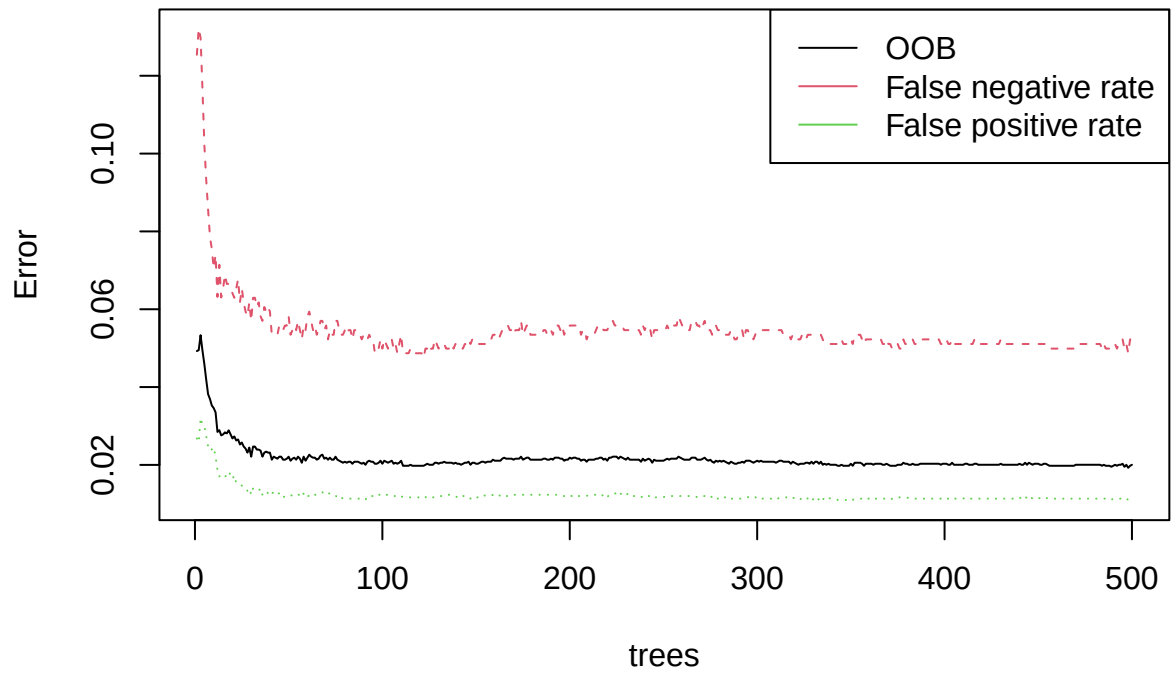


Figure 2: OOB against the number of trees

Figure 2 shows that 400 trees seems to be good enough. Hence, we refit the random forest under ntree = 400.

```
final.rf1 = randomForest(y~., data = pc.train,
                         split = "deviance", mtry = opt.m, n.tree = 400)
# importance(rf)
varImpPlot(final.rf1, main = "")
```

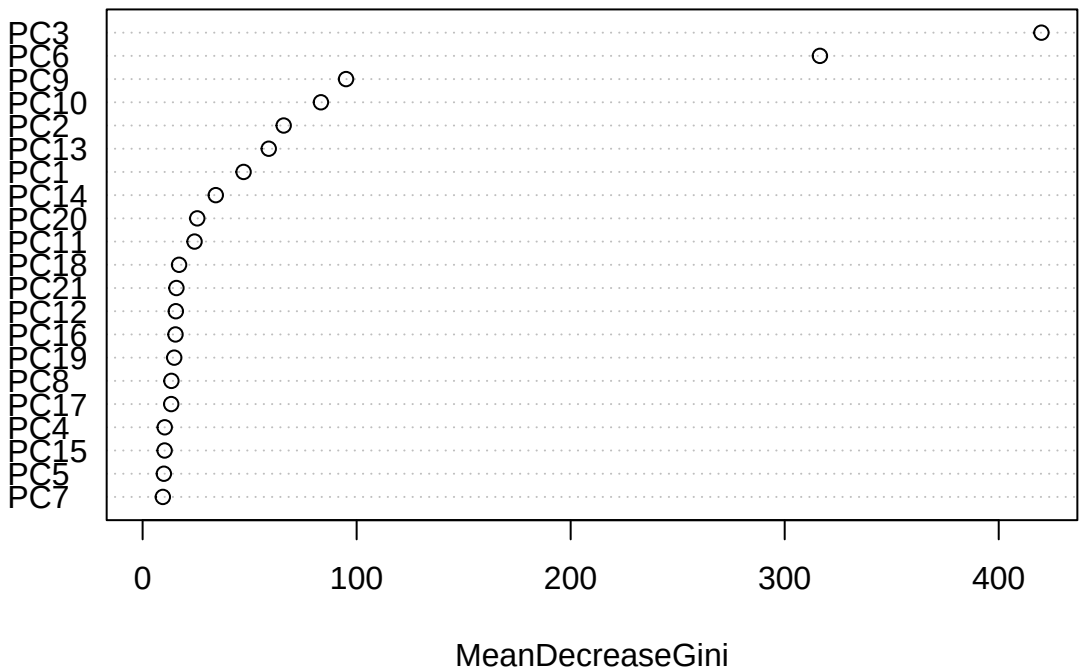


Figure 3: Important plot for predictors

Figure 3 shows the importance plot, from which one can see that PC3, PC6 and PC9 seem to be the most significant predictors since they on average reduce more Gini index than than others do.

In hope of understanding the meaning of each PC and thereby explain why PC3, PC6 and PC9 seem to be the most useful predictors, we go back to find out the loading vectors by which matrices can be reconstructed which would show the positions of information contained in each PC.

```
p1 = pheatmap(matrix(V[,3], nrow = 100), cluster_rows = FALSE,
              cluster_cols = FALSE,
              main = "Loadings for PC3", border_color = NA)
p2 = pheatmap(matrix(V[,6], nrow = 100), cluster_rows = FALSE,
              cluster_cols = FALSE,
              main = "Loadings for PC6", border_color = NA)
p3 = pheatmap(matrix(V[,9], nrow = 100), cluster_rows = FALSE,
              cluster_cols = FALSE,
              main = "Loadings for PC9", border_color = NA)
dump(c("p1", "p2", "p3"), "heatmap1.Rdata")
```

```
plot_list=list()
plot_list[['p1']]=p1[[4]]
plot_list[['p2']]=p2[[4]]
plot_list[['p3']]=p3[[4]]
grid.arrange(grobs=plot_list,ncol=2)
```

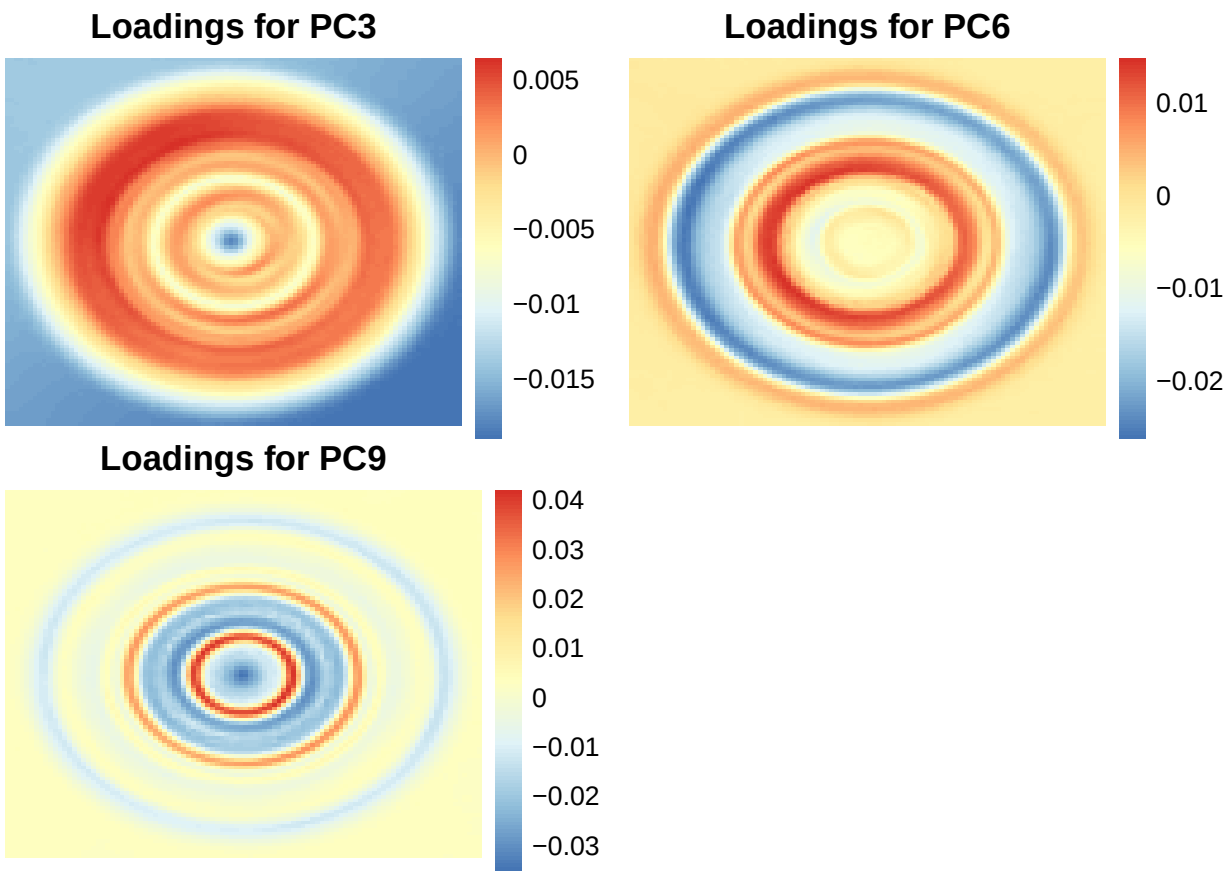


Figure 4: Loadings for the PCs

Figure 4 shows the loading of each pixel of the image for PC3, PC6 and PC9. One can see that PC3 basically represents the background information, such as light, shadow and so forth, while PC6 and PC9 represent the outside-cirle and inside-circle information respectively. Such result is quite meaningful and of high interpretability for its coincidence with a theory in perceptual psychology which says that visual processing is involved with combining information from object contours and background information.

b. Support vector machine with kernel = "linear"

To train a svm, we specify a linear kernel where there is only a tuning parameter "cost" to select, which determine the cost of a violation to the margin. We select it via 5-fold cross validation. Throughout the procedure, cost is searched among the 10 points which are equally spaced between 0.2 and 6.

```
params1 = seq(0.2, 6, length = 10) #cost
```

```
params1 = seq(0.2, 2, length = 10) #cost
cv.errors = array(NA, dim = c(k, length(params1)))
for(i in 1:length(params1)){
  print(params1[i])
  for(j in 1:k){
      fit = svm(y~., data =  pc.train, kernel = "linear",
                  subset = folds != j, cost = params1[i])
      pred = predict(fit, newdata = pc.train[folds == j,])
      confuse = table(pc.train[folds == j,]$y, pred)
      acc = sum(diag(confuse))/sum(confuse)
```

```
      cv.errors[j, i] = acc
  }
}
dump("cv.errors", "Q1_svm2_cv_err.RData")
```

```
cv = apply(cv.errors*matrix(rep(folds.n, length(params1)), nrow = k), 2, sum)/nrow(pc.train)
se = sqrt(apply((cv.errors - matrix(rep(cv, k), byrow = TRUE, nrow = k))^2, 2, sum)/(k*(k-1)))
matplot(params1,
        cbind(cv, cv-se, cv+se), type = "l", lty = c(1, 2, 2), col = "black",
        xlab = "cost", ylab = "cv Accuracy")
abline(v = params1[which.max(cv)], col = "red")
legend("topleft", c("cv Accuracy", "CI"), lty = 1:2)
```

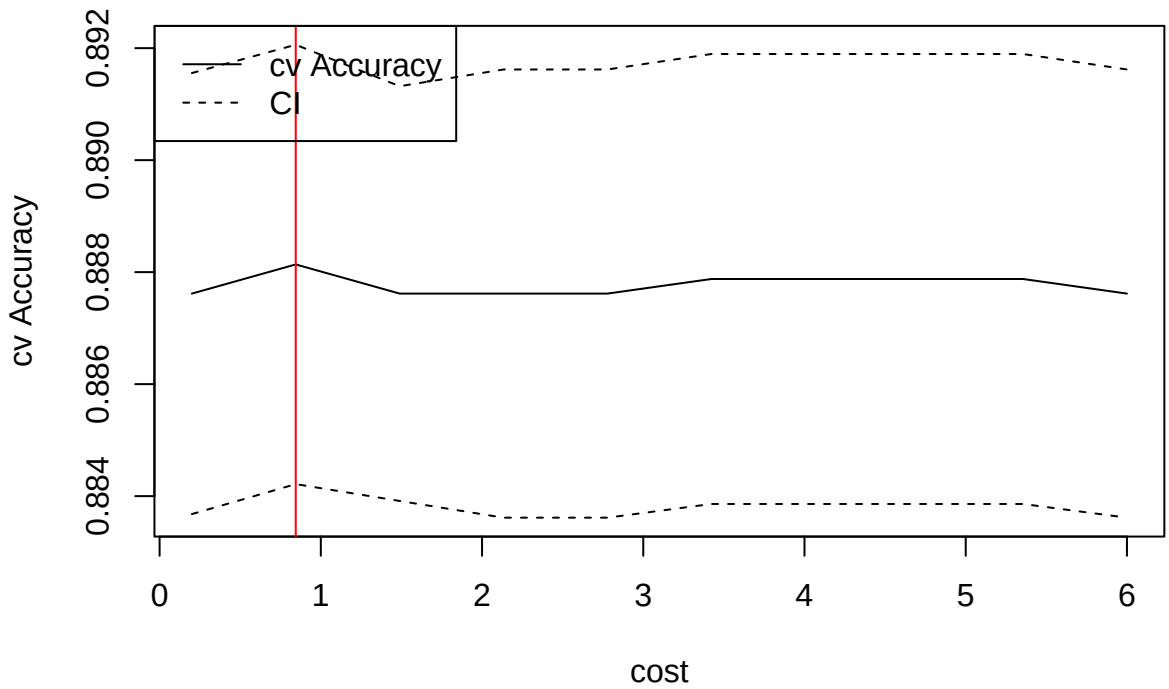


Figure 5: Cross validation for the tuning parameter cost

Figure 5 suggests that it might be better to set cost at 0.8444444 for the highest accuracy estimate. We thus fit again the svm under cost = 0.8444444 using the whole training data.

```
svm2 = svm(y~., data = pc.train, kernel = "linear",
            cost = params1[which.max(cv)], probability = TRUE)
```

c. Support vector machine with kernel = "radial"

To train a svm, we specify a radial kernel where *gamma* is a tuning parameter to be selected. In addition, there is also a tuning parameter "cost" to select, which determine the cost of a violation to the margin. We

select them via 5-fold cross validation. Throughout the procedure, cost is searched among the 10 points which are equally spaced between 0.2 and 6, and γ is searched among the 10 points which are equally spaced between 0.02 and 0.3.

```
params1 = seq(0.2, 6, length = 10) #cost
params2 = seq(0.02, 0.3, length = 10) #gamma
```

```
params1 = seq(0.2, 6, length = 10) #cost
params2 = seq(0.02, 0.3, length = 10) #gamma
cv.errors = array(NA, dim = c(k, length(params1), length(params2)))
for(i in 1:length(params1)){
  for(l in 1:length(params2)){
    print(c(params1[i], params2[l]))
    for(j in 1:k){
        fit = svm(y~., data =  pc.train, kernel = "radial",
                    subset = folds != j, cost = params1[i], gamma = params2[l])
        pred = predict(fit, newdata = pc.train[folds == j,])
        confuse = table(pc.train[folds == j,]$y, pred)
        acc = sum(diag(confuse))/sum(confuse)
        cv.errors[j, i, l] = acc
    }
  }
}
dump("cv.errors", "Q1_svm1_cv_err.RData")
```

The matrix shown in Figure 6 displays the result of cross validation. Each row of it denotes each distinct value of cost, while each column of it denotes each distinct value of γ. Each entry of the matrix displays the test accracy estimate calculated by averaging across the test accuracies obtained for each fold under the corresponding setting of cost and γ.

```
cv = apply(cv.errors*array(matrix(rep(folds.n, length(params1)), nrow = k),
                           dim = c(k, length(params1), length(params2))),
                           c(2, 3), sum)/nrow(pc.train)
rownames(cv) = round(params1, 3); colnames(cv) = round(params2, 3)
pheatmap(cv, cluster_rows = FALSE, cluster_cols = FALSE, main = "cv Accuracy")
```

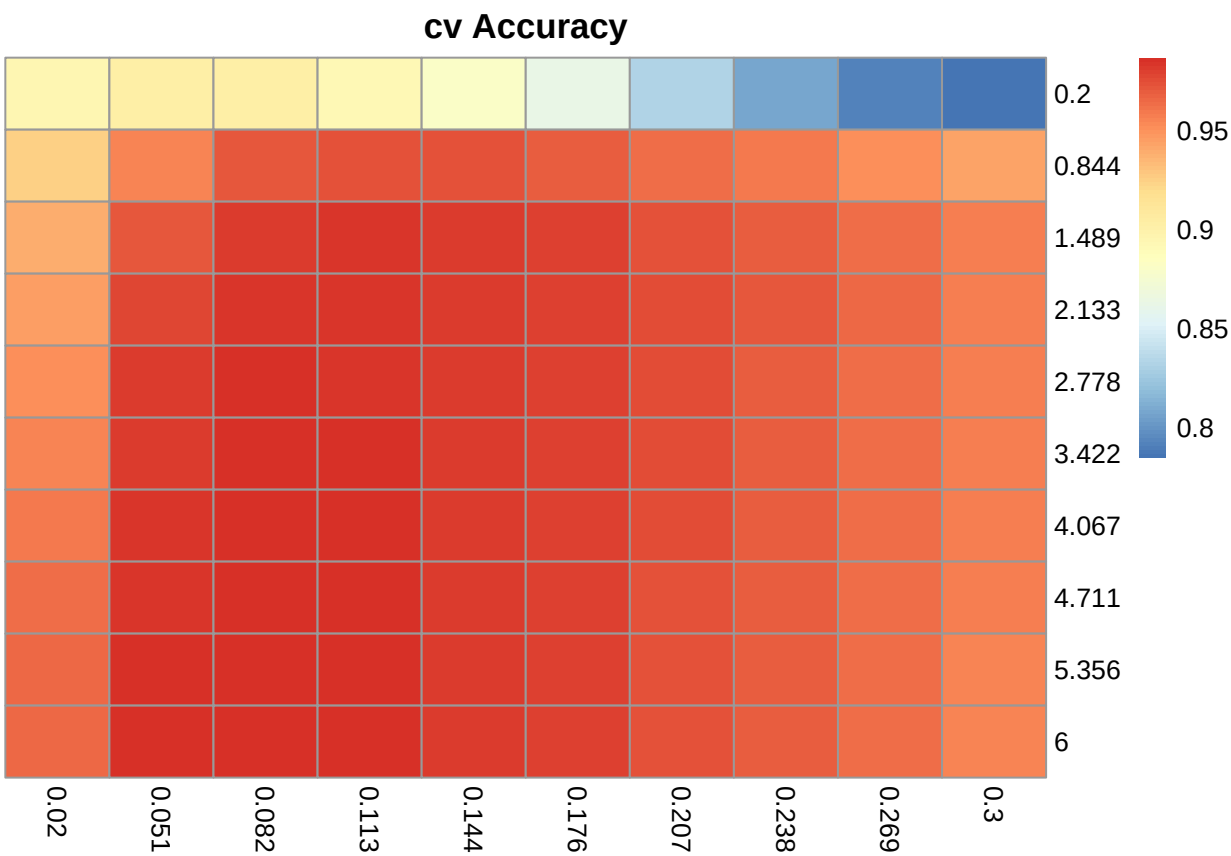


Figure 6: Loadings for the PCs

```
opt.params = c(params1[ifelse(which.max(cv) %% 10 == 0, 10, which.max(cv) %% 10)],
            params2[ifelse(which.max(cv) %% 10 == 0, which.max(cv) %/% 10,
                           which.max(cv) %/% 10 + 1)])
```

The figure suggests that it might be better to set cost at 3.422 and γ at 0.082 for the highest accuracy estimate. We thus fit again the svm under cost = 3.422 and $\gamma = 0.082$ using the whole training data.

```
svm1 = svm(y~., data =  pc.train, kernel = "radial",
            cost = opt.params[1], gamma = opt.params[2], probability = TRUE)
```

0.1.6 Evaluate the prediction performance via testing error on testing set

For convenience, we abbreviate "support vector machine with linear kernel" as "LSVM" and "support vector machine with radial kernel" as "RSVM" in the following discussion.

```
test.summary = matrix(NA, nrow = 1, ncol = 3)
colnames(test.summary) = c("Random Forest", "LSVM", "RSVM")
rownames(test.summary) = c("Accuracy")
models = list(final.rf1, svm2, svm1)
ROC = list(); auc = rep(NA, 3)
pc.test = as.matrix(scale.data[test, -10001]) %*% V[,1:21]
pc.test = as.data.frame(pc.test)
```

```r
pc.test$y = data[test,]$y
for(i in 1:length(models)){
  fit = models[[i]]
  colnames(pc.test)[-22] = paste("PC", 1:21, sep = "")
  # predict
  pred = predict(fit, newdata = pc.test)
  confuse = table(pc.test$y, pred)
  # test acc
  acc = sum(diag(confuse))/sum(confuse)
  test.summary[,i] = acc
  # test roc
  if(i == 1){
    pred = predict(fit, newdata = pc.test, type = "prob")[,2]
    rocobj = roc(pc.test$y, pred)
    auc[i] = round(auc(pc.test$y, pred),4)
  }else{
    pred = predict(fit, newdata = pc.test, probability = TRUE)
    pred = attr(pred,"probabilities")[,1]
    rocobj = roc(pc.test$y, pred)
    auc[i] = round(auc(pc.test$y, pred),4)
  }
  ROC[[i]] = (ggroc(rocobj, colour = 'steelblue', size = 2) +
  ggtitle(paste0(colnames(test.summary)[i], ' ROC Curve ', '(AUC = ', auc[i], ')'))) %>%
  +theme_bw()+theme(plot.title = element_text(size = 6))
}
```

```r
kable(as.data.frame(round(test.summary, 4)), booktabs = TRUE, linesep = "",
      align = "c", caption = "Summary of testing error for each model") %>%
  kable_styling(latex_options = c("hold_position"))
```

Table 1: Summary of testing error for each model

	Random Forest	LSVM	RSVM
Accuracy	0.9802	0.8772	0.974

Table 1 shows the cv Accuracy of each model. One can see the accuracies are almost 100% for random forest and RSVM, suggesting that these two classifiers are quite comparable, both of which perform excellently in this classification task. However, LSVM, which is less complex and flexible than RSVM, fails to compete with random forest and RSVM for its much lower accuracy.

```r
grid.arrange(grobs = ROC, ncol=3)
```

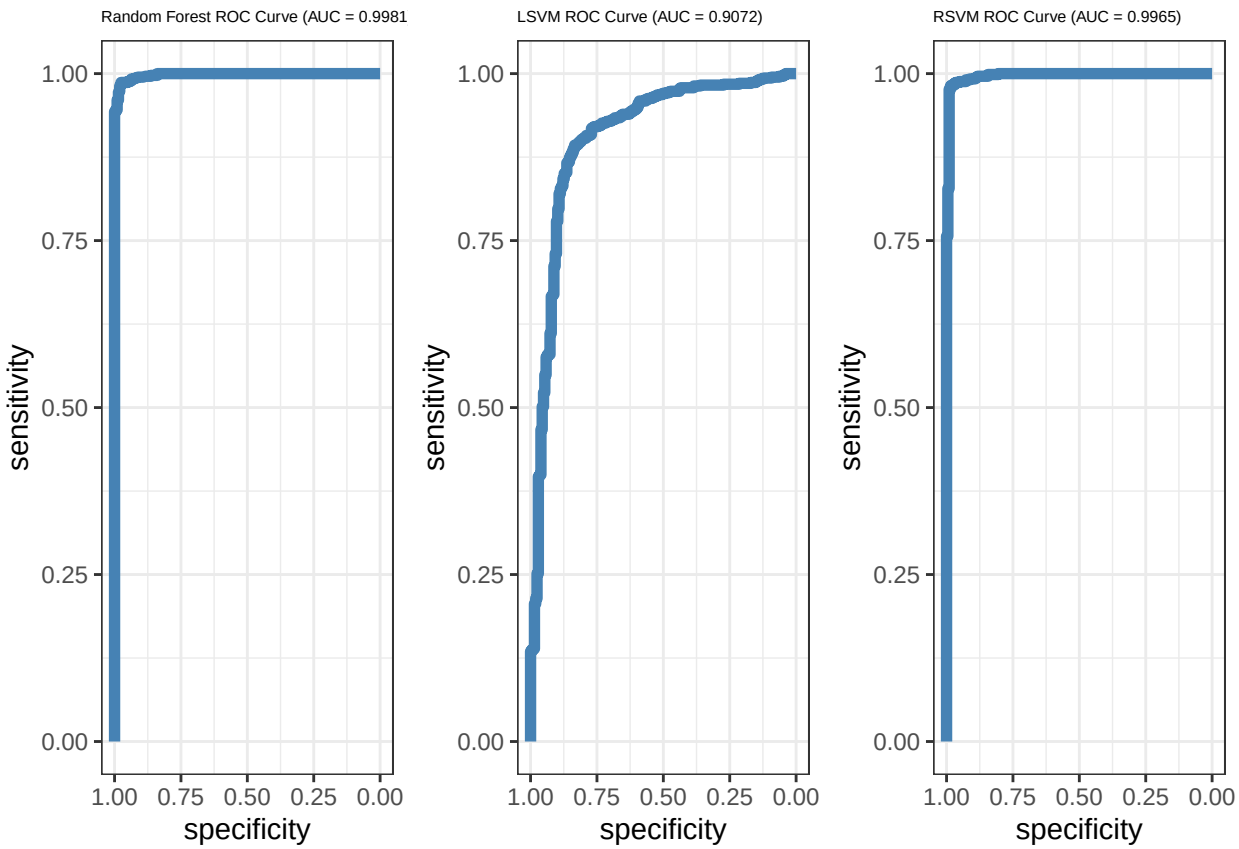


Figure 7: ROC curve for random forest and svm

Figure 7 further shows the ROC curves for the three classifiers with AUCs provided, which also reveal how perfect the random forest an RSVM are since the AUC of them are 99.81% and 99.65%.

0.1.7 A brief analysis summary.

1. It is found in Figure **??** and Figure 4 that PC3, PC6 and PC9 seem to be the most useful predictor in this classification task. In addition, they are meaningful and of high interpretability.

2. The performance of random forest and svm are comparable. Both of them perform extremely well with their classification accuracies being almost 100%. In the contrast, LSVM performs much worse than the two formers perhaps due to its less flexibility.

0.1.8 (Bonus) Can defect items (image data) be clustered with patterns (sub-types)? Explore your findings.

0.1.8.1 Method 1 First, we scale the whole data and conduct principal component analysis on the scaled data. Then we use only the part of defect items in the subsequent analysis. There are two reasons regarding why we consider principal component analysis. The first reason is out of the findings in the last question where we noted that principal component analysis seems to be a convenient and reasonable way for the data preprocessing, while the second reason is out of the fact that principal component analysis seems to indeed provide reasonable result which enjoys high interpretability.

```
scale.def.data = scale(data[data$y == 1,-10001])
```

```
X = scale.def.data
eigen.decomp2 = eigen(cov(X))
eigen.decomp2$values = eigen.decomp$values[1:153]
eigen.decomp2$vectors = eigen.decomp$vectors[,1:153]
dump("eigen.decomp2", "eigen.decomp2.Rdata")
```

Figure 8 shows the PCs which we found meaningful, say PC6 and PC10. Why we say so is because it seems that these two principal components basically represent the two distinct information from inside-circle and outside-circle respectively, by which it could become easier to interpret the result if we cluster the defect items with respect to such two principal components.

```
p1 = pheatmap(matrix(V[,6], nrow = 100), cluster_rows = FALSE,
              cluster_cols = FALSE,
              main = "Loadings for PC6", border_color = NA)
p2 = pheatmap(matrix(V[,10], nrow = 100), cluster_rows = FALSE,
              cluster_cols = FALSE,
              main = "Loadings for PC10", border_color = NA)
dump(c("p1", "p2"), "heatmap2.Rdata")
```

```
plot_list=list()
plot_list[['p1']]=p1[[4]]
plot_list[['p2']]=p2[[4]]
grid.arrange(grobs=plot_list,ncol=2)
```

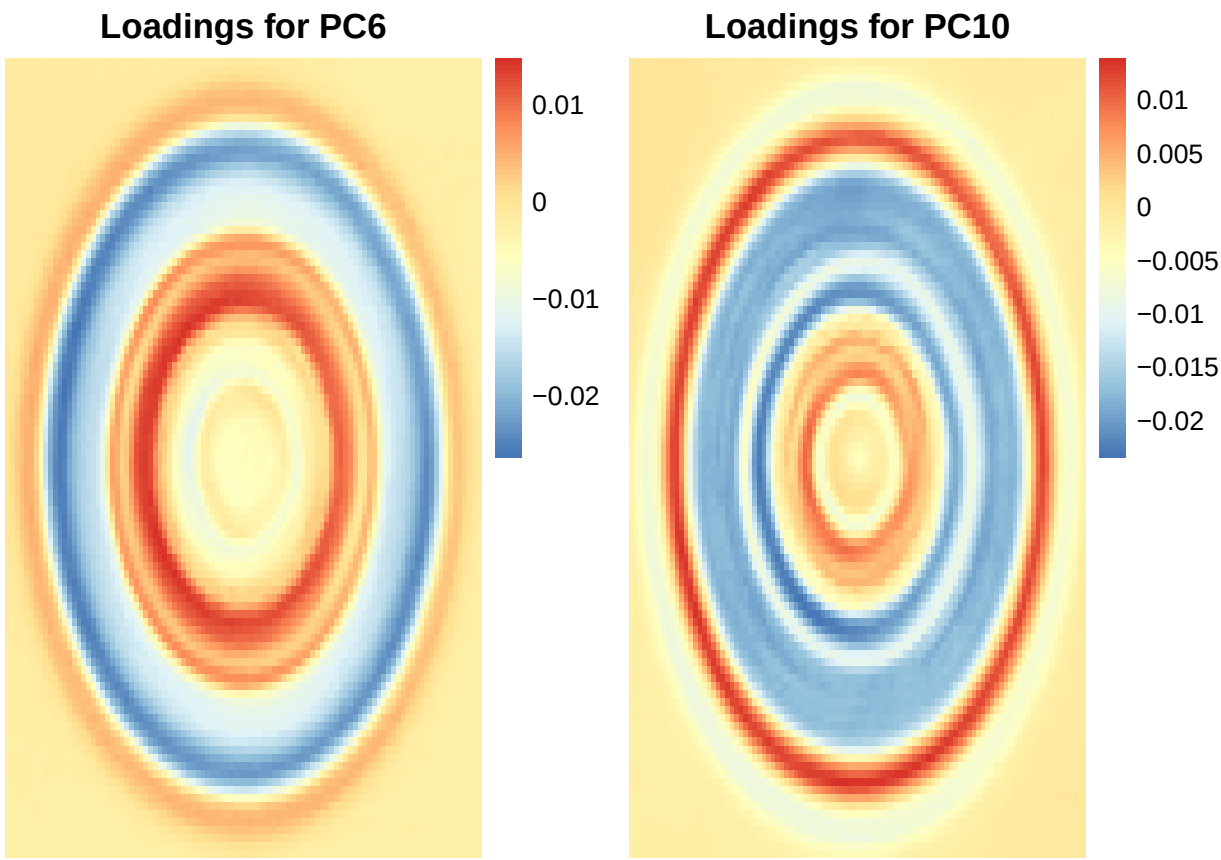


Figure 8: Loadings for the PCs

```
pc.data = as.data.frame(as.matrix(scale.def.data) %*% V[,1:50])
```

To find whether the defect items can be clustered with patterns represented by PC6 and PC10, we consider to apply kmedoids in the following analysis. Kmedoids is chosen simply because that it was the one that seems to lead to the best result after several times of trials and errors. Figure 9 shows the result of elbow method by which we determine a suitable number of clusters k.

```
PC = c(6, 10)
fviz_nbclust(pc.data[, PC],
             FUNcluster = pam,
             method = "wss",
             k.max = 8
             ) %>%
+ geom_vline(xintercept = 4, linetype = 2)
```

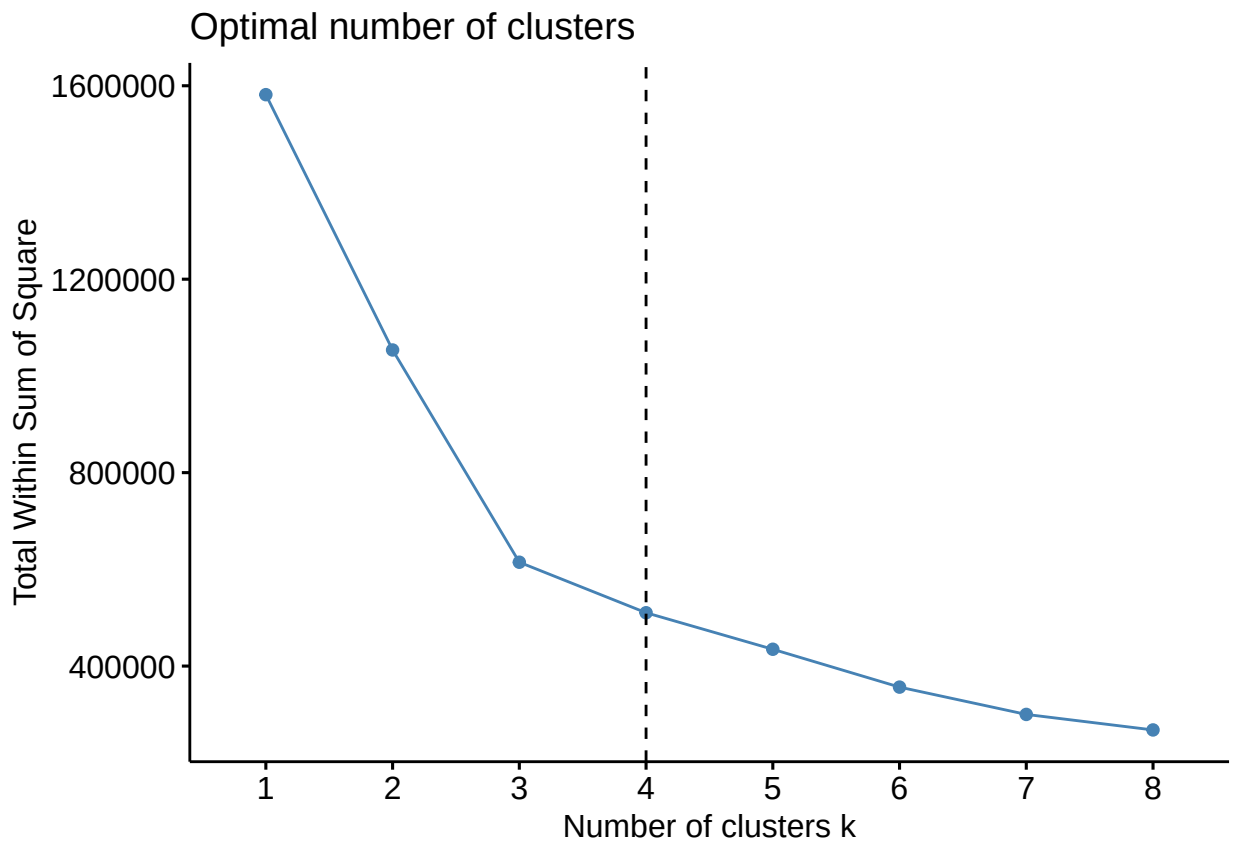


Figure 9: Elbow Method for K-Medoids

From Figure 9, one can see that the elbow seems to appear at $k = 4$. Hence, we choose to set the number of clusters to be 4.

```
set.seed(111024513)
kmedoid.cluster = pam(pc.data[, PC], k=4, nstart = 20)
fviz_cluster(kmedoid.cluster,
             data = pc.data[, PC],
             geom = c("point","text"),
             frame.type = "norm") + theme_bw()
```

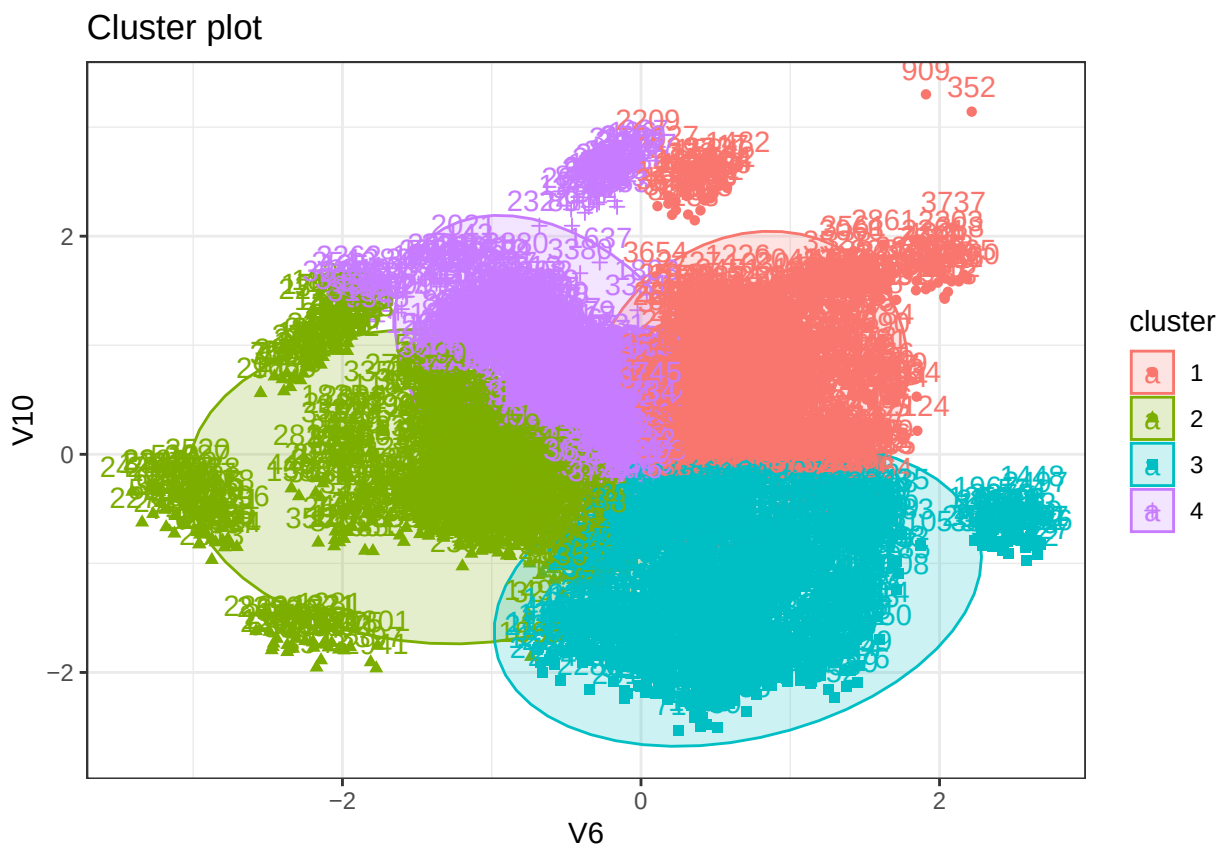


Figure 10: Visualization of clustering

Figure 10 shows the result of clustering by kmedoids. It seems to be quite satisfactory since there does not seem to be much overlap among the clusters. Moreover, one can see that cluster 1 is characterized by relatively lower values of PC6 and PC10, which corresponds to darker appearance at the inside circle and outside circle according to the loadings shown in Figure 8. Similarly, one can find that cluster 2 is characterized by darker appearance at outside circle while brighter appearance at the inside circle, cluster 3 is characterized by brighter appearance at the outside and inside circle, and cluster 4 is characterized by brighter appearance at outside circle while darker appearance at inside circle.

To interpret the clustering result, we then find out the images of the items that are clustered into each of the clusters. We simply sampled two for each cluster and shows them in Figure @(fig:img), where the first column consists of the two sampled items for cluster 1, the second column consists of the two for cluster 2, and so forth. Though the pattern might not be quite obvious, we can still find it somewhat consistent with the characters for each of the clusters as described above.

```
par(mfrow = c(5, 4))
set.seed(20000212)
sam = sample(1:200, 5)
for(j in 1:length(sam)){
  s = sam[j]
  for (i in 1:4) {
    show_image((data[data$y == 1,])[kmedoid.cluster$cluster == i, -10001][j,],
           col = gray(1:20/20))
    box(); title(paste("Defect Item ", j," (cluster = ", i,")", sep = ""), cex.main = 3)
  }
```

```
}
```

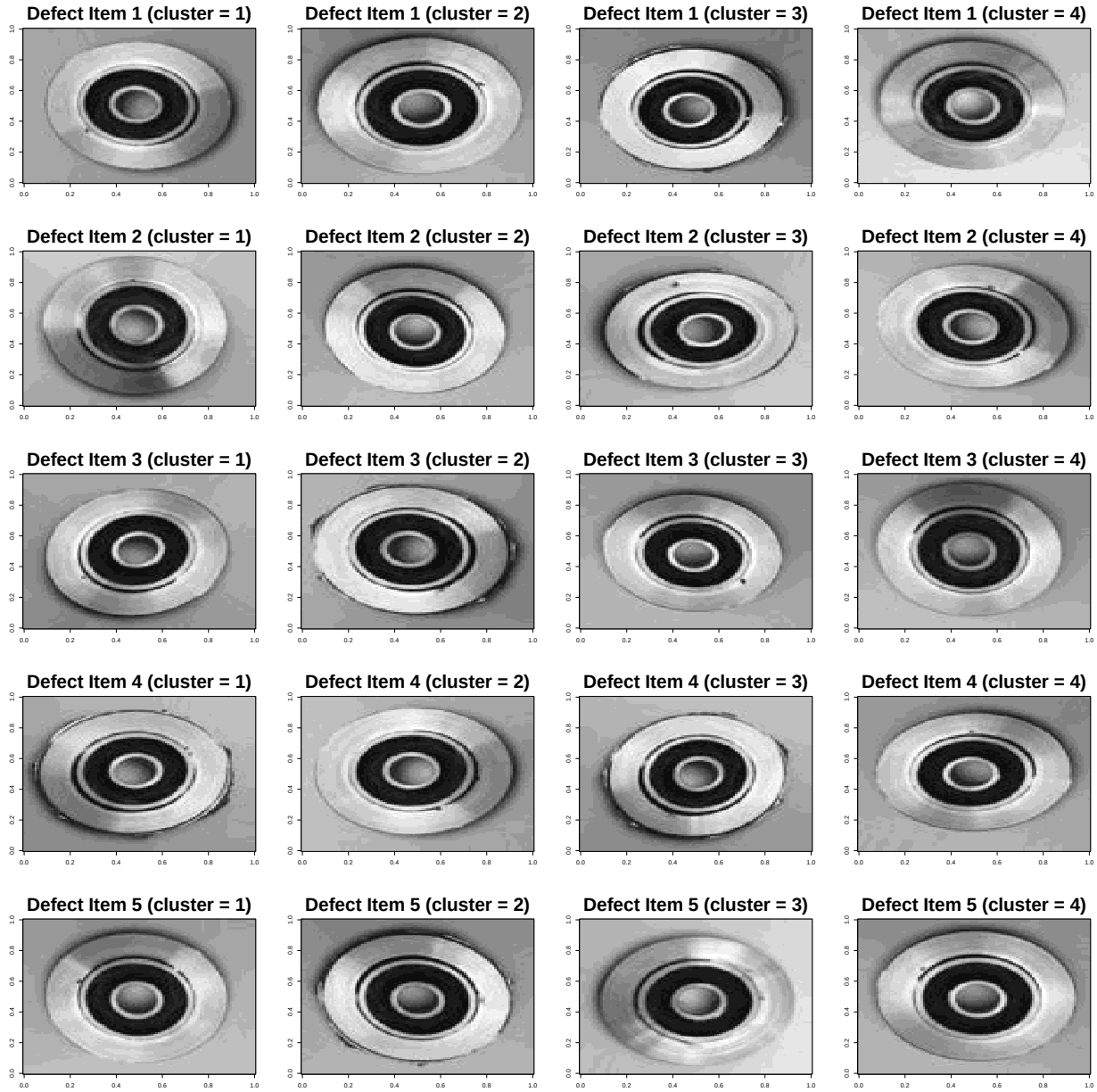


Figure 11: Original images

0.1.8.2 Method 2 Here is also an alternative approach which should be credited to Mr.Kang (康暐郡). His main idea is to use tsne to find a good space to represent the data with an suitable set of tuning parameters: `k = 2, perplexity = 30, initial_dims = 3, max_iter = 500`. What I have done is merely finding a clustering method which might somehow produce an satisfactory result. However, the clustering method still should be credited to Mr.Kang for much of his valuable suggestions and the informative discussion with him.

```
tsne_3 = tsne(as.matrix(data.def), k = 2, perplexity = 30, initial_dims = 3, max_iter = 500)
```

Using the result of tsne, we chose to perform hierarchical clustering with `linkage = median`. Since I observed that the defects seem to usually appear either on the outer contour or the inner contour, I decided to set the number of clusters into 2. Figure 12 and 13 shows the result of clustering which look satisfactory.

```
set.seed(111024513)
hcluster = hcut(tsne_3, k=2, hc_method = "median")
plot(hcluster)
```

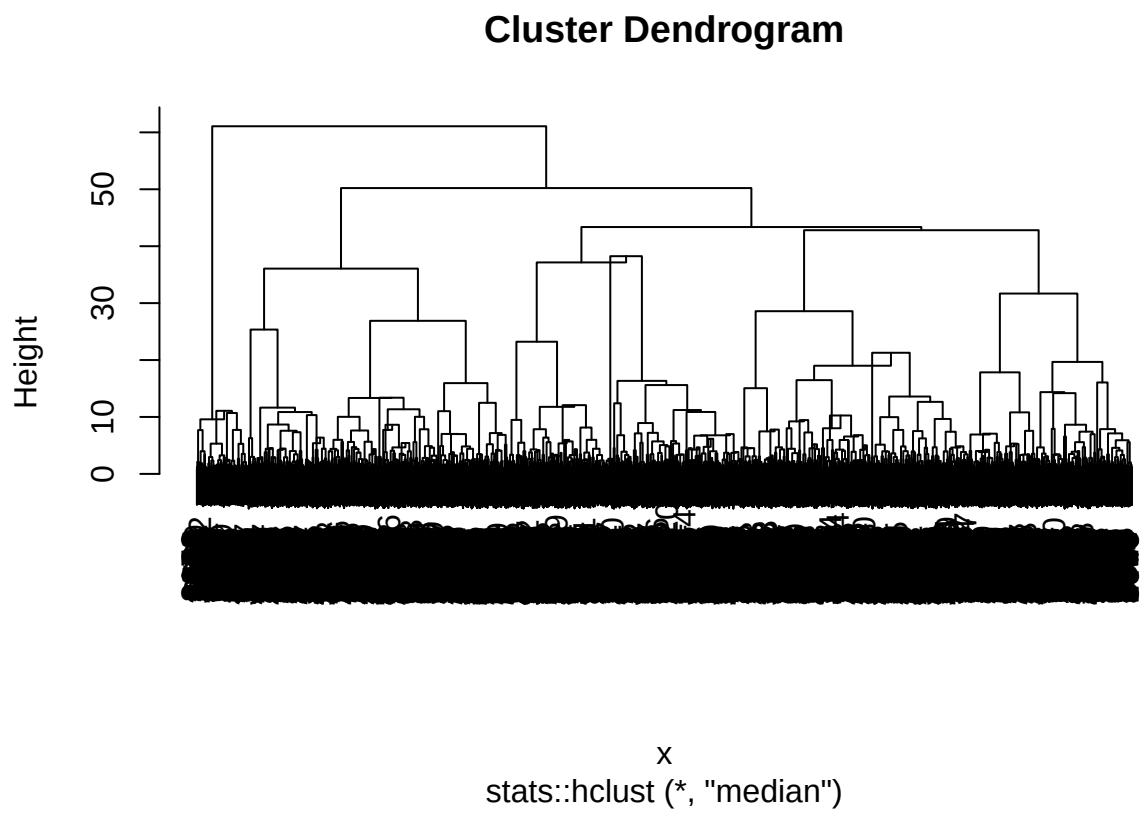


Figure 12: Result of hierarchical clustering

```
plot(pc.data[,1], pc.data[,2], col = hcluster$cluster, xlab = "PC1", ylab = "PC2")
legend("topright", c("cluster 1", "cluster 2"), pch = 1, col = 1:2)
```

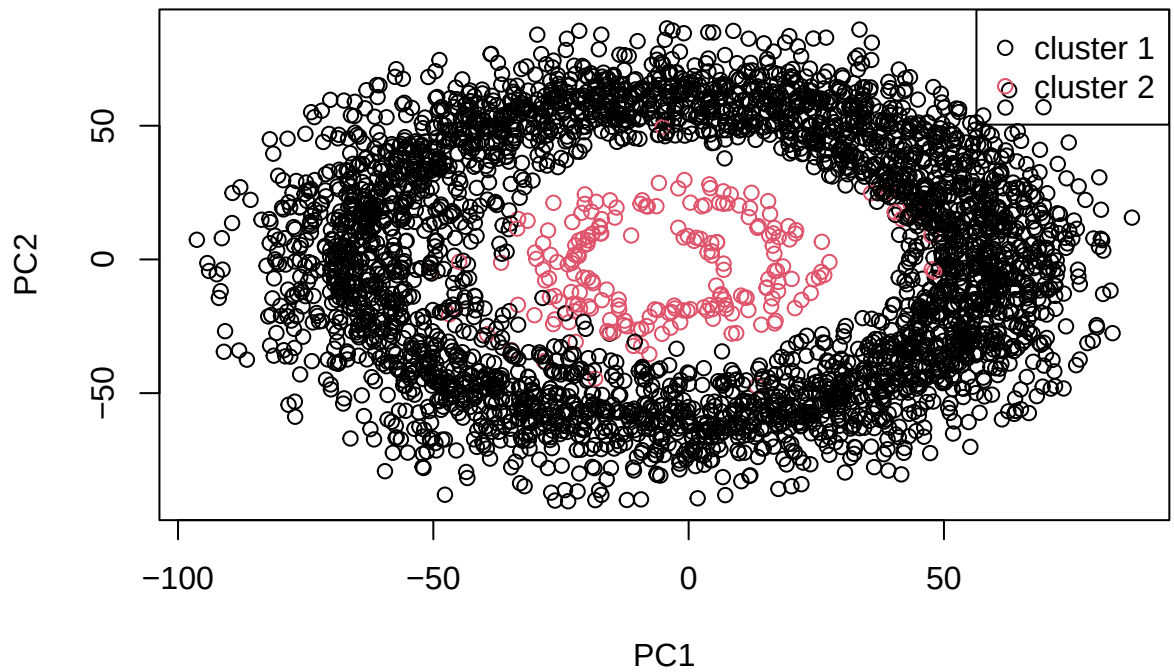


Figure 13: Visualization of clustering

To interpret such result of clustering, I use the same method as described in method 1 to pull out 5 original images for the first and the second cluster respectively. Figure **??** shows those images from which we can see obvious difference between the two clusters, that is, the defect items belonging to cluster 1 tend to have defects on the outer contour while those belonging to cluster 2 tend to have defects on the inner contour.

```
par(mfrow = c(5, 2))
set.seed(11111)
s = sample(1:100, 5)
for(j in s){
  for (i in 1:2) {
    show_image((data[data$y == 1,])[hcluster$cluster == i, -10001][j,],
           col = gray(1:20/20))
    box(); title(paste("Defect Item "," (cluster = ", i,")", sep = ""),
                 cex.main = 3)
  }
}
```

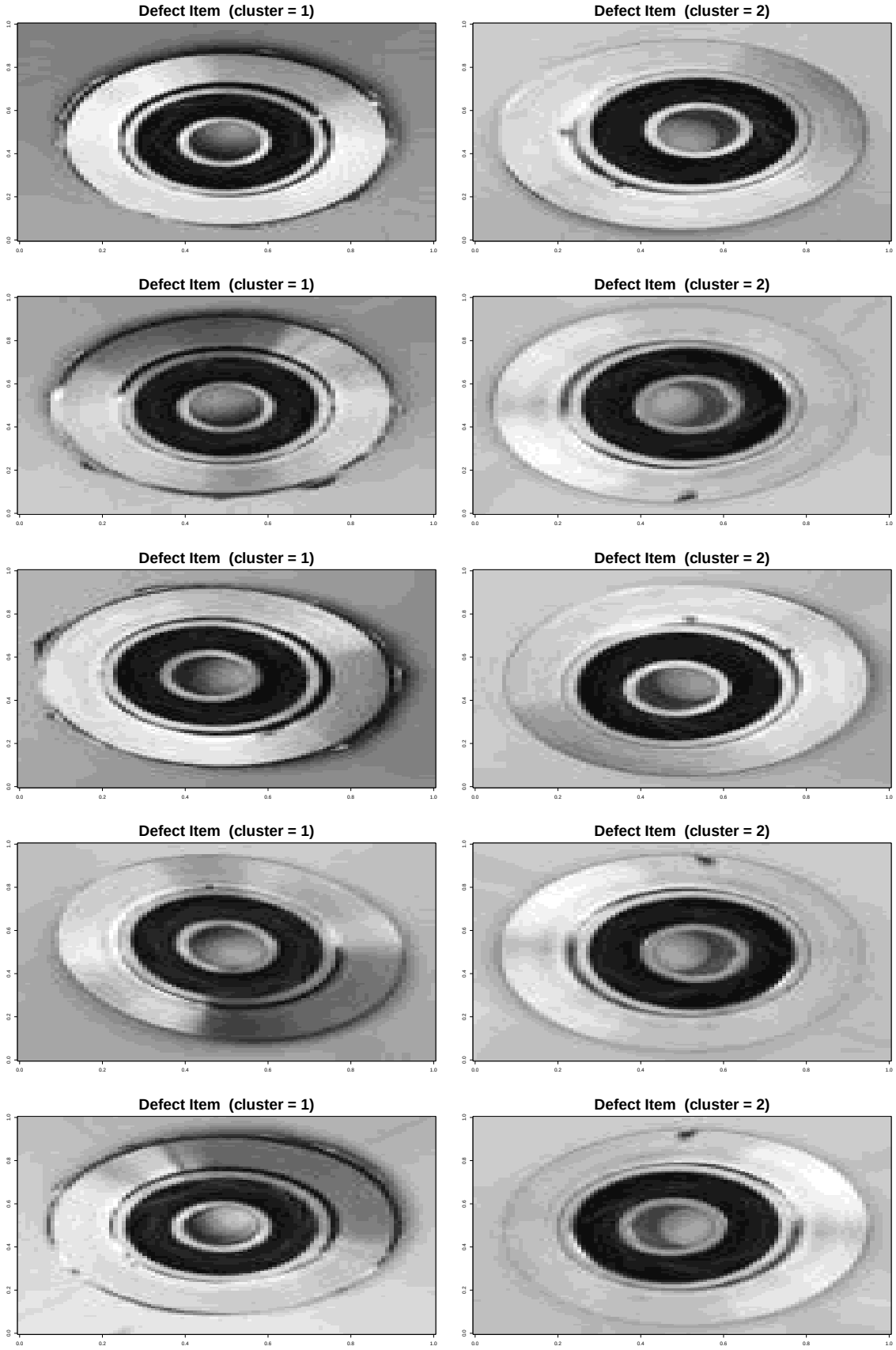


Figure 14: Original images

0.2 Problem 2: Forecast for SP500 returns

This problem concerns about 2 tasks on forecasting the return for SP500:

- Task 1 (regression): forecast the daily return value;
- Task 2 (classification): forecast the trend ("up" if return greater than zero; or "down" otherwise).

The data can be retrieved from yahoo into R via the function `getSymbols` in `quantmod` package. In this homework, we take the data before 2021/12/31 as the training data; and the data after 2022/1/1 as the testing data.

You may replace the SP500 returns by any other return series of your interest for this problem.

0.2.1 Your analysis should include the following:

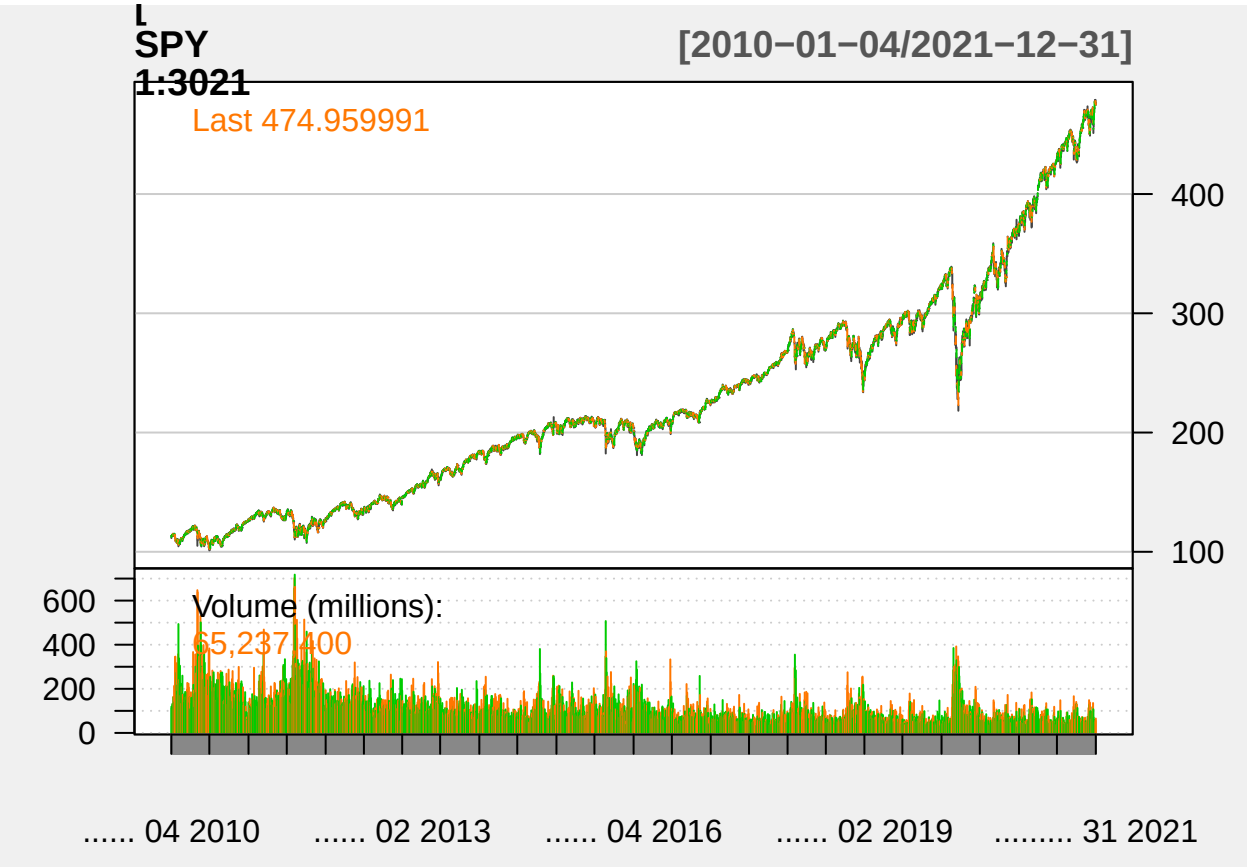
- The training procedures of the predictive models for both tasks. (training period: before 2022)
- Summarize the prediction performance in the testing period (2022/1/1-2022/12/31).

0.2.2 Get data

```
#install required package

library(quantmod)

getSymbols("SPY", src = "yahoo", from = as.Date("2010-01-01"), to = as.Date("2022-12-31"))
```

```
## [1] "SPY"
```

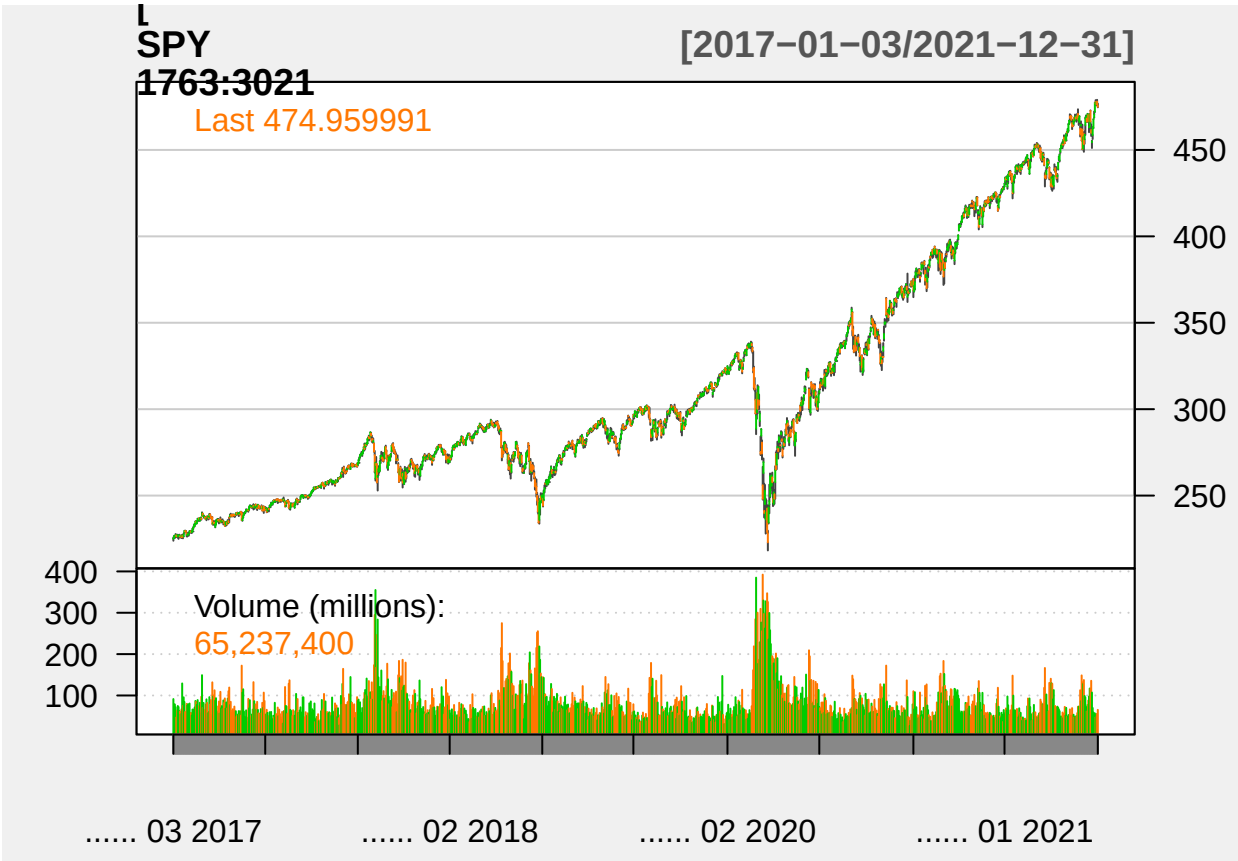
```
chartSeries(SPY[1:3021], theme = chartTheme("white"))
```



上圖為 SPY 股價趨勢圖，可以看到整體股價除了中段有一些下降趨勢外，整體股價皆呈現上升趨勢；交易量圖與股價趨勢圖也可以看出在交易量多的時候股價大多都是呈現下跌，此也符合若市場出現負面資訊導致投資人心理預期股價下跌時，會趕緊將手頭持倉賣出，造成交易量變得比平常還高。

此題欲預測 2022 年以後的每日報酬率，考慮到股價為一時序資料，離太遠的資料相關性就越低，對預測幫助亦越低，加上股價的不穩定性很高，所以我們想要擷取 2017 年至 2021 年底的資料當作訓練集。因這段資料包含了三次突發的下降趨勢，其中最大下降幅度的時期正值疫情爆發初期，當時造成的全球性經濟衰退導致此現象的發生；另外此段訓練集資料亦包含了疫情後的經濟緩慢回穩、股價的上升趨勢。所以我們認為這段包含不同的 pattern 的資料集比較能夠代表股價的不穩定性，且離 2022 年也不至於太遠，對於後疫情的 2022 年或許會有較好的預測效果。(訓練集資料的股價趨勢圖如下)

```
chartSeries(SPY[1763:3021], theme = chartTheme("white"))
```



決定訓練集資料後，我們先算出每日報酬率、至 time lag 5 的報酬率、至 time lag 5 的交易量，打算以這些變數來進行後續的分析。

```
# compute daily returns:
rr <- exp(diff(log(SPY$SPY.Close)))-1
Direction <- ifelse(rr >= 0, "Up", "Down")
names(Direction) <- "Direction"

# create lag variables (you may use more lags or other info from SPY object)
rr.Lag <- Lag(rr, 1:5)
Volume.Lag <- Lag(SPY$SPY.Volume,1:5)

#For Task 1 (respones variable: rr):
SPY.return <- data.frame(rr, rr.Lag, Volume.Lag/10^9) #rescale Volume such that all scales are comparab
names(SPY.return) <- c("r", "r.Lag.1", "r.Lag.2", "r.Lag.3",  "r.Lag.4", "r.Lag.5",
                            "v.Lag.1", "v.Lag.2", "v.Lag.3",  "v.Lag.4", "v.Lag.5")

SPY.return = na.omit(SPY.return) #remove NA's
```

另外也針對 classification 造另一個資料集。

```
#For Task 2 (response variable: Direction):
SPY.trend <- data.frame(Direction, rr.Lag, Volume.Lag/10^9)
names(SPY.trend) <- c("Direction", "r.Lag.1", "r.Lag.2", "r.Lag.3",  "r.Lag.4", "r.Lag.5",
                        "v.Lag.1", "v.Lag.2", "v.Lag.3",  "v.Lag.4", "v.Lag.5")
```

```r
SPY.trend = na.omit(SPY.trend) #remove NA's
SPY.trend$Direction <- factor(SPY.trend$Direction)
```

再來我們可以看一下 return 資料集的各變數平均值和標準差，不意外的是其實報酬率相關和交易量相關的變數各自的 mean 和 standard deviation 都十分相近 (因為 lag)。

```r
apply(SPY.return,2,mean)
```

```
##            r      r.Lag.1      r.Lag.2      r.Lag.3      r.Lag.4      r.Lag.5
## 0.0004309984 0.0004322325 0.0004277400 0.0004328377 0.0004342607 0.0004333100
##      v.Lag.1      v.Lag.2      v.Lag.3      v.Lag.4      v.Lag.5
## 0.1182838792 0.1183020763 0.1183205024 0.1183402318 0.1183560685
```

```r
apply(SPY.return,2,sd)
```

```
##          r    r.Lag.1    r.Lag.2    r.Lag.3    r.Lag.4    r.Lag.5    v.Lag.1
## 0.01114040 0.01114028 0.01113615 0.01113407 0.01113381 0.01113349 0.07146461
##    v.Lag.2    v.Lag.3    v.Lag.4    v.Lag.5
## 0.07145911 0.07145464 0.07144512 0.07143789
```

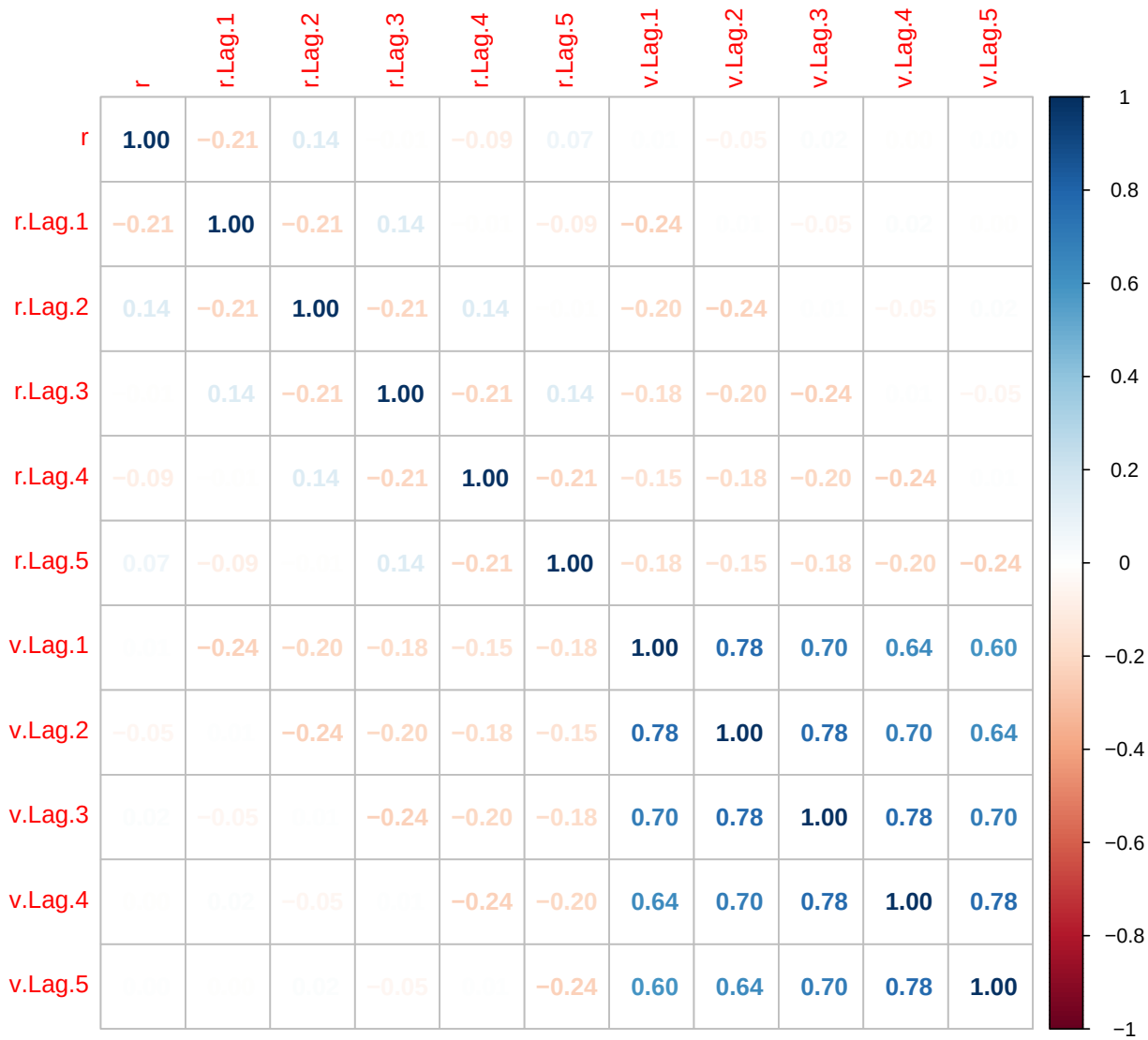
切出訓練集和測試集的資料，並造出時序版本的 5-fold CV 方式的 index。

```r
splitindex <- which(rownames(SPY.return) == "2021-12-30")
SPY.return.train <- SPY.return[1762:splitindex,]
SPY.return.test <- SPY.return[splitindex:nrow(SPY.return),]
SPY.trend.train <- SPY.trend[1762:splitindex,]
SPY.trend.test <- SPY.trend[splitindex:nrow(SPY.return),]
```

```r
cvindex.1st <- 1:(nrow(SPY.return.train)-150*5)
cvindex.2nd <- 1:(nrow(SPY.return.train)-150*4)
cvindex.3rd <- 1:(nrow(SPY.return.train)-150*3)
cvindex.4th <- 1:(nrow(SPY.return.train)-150*2)
cvindex.5th <- 1:(nrow(SPY.return.train)-150*1)
validindex.1st <- (length(cvindex.1st)+1):length(cvindex.2nd)
validindex.2nd <- (length(cvindex.2nd)+1):length(cvindex.3rd)
validindex.3rd <- (length(cvindex.3rd)+1):length(cvindex.4th)
validindex.4th <- (length(cvindex.4th)+1):length(cvindex.5th)
validindex.5th <- (length(cvindex.5th)+1):nrow(SPY.return.train)
cvindex <- list(cvindex.1st, cvindex.2nd, cvindex.3rd, cvindex.4th, cvindex.5th)
validindex <- list(validindex.1st, validindex.2nd, validindex.3rd,
                   validindex.4th, validindex.5th)
```

最後看一下訓練集資料中各變數的相關性可以發現，每日報酬率和 lag1 報酬率呈現些微負相關，但和 lag2 呈現些微正相關，與其他變數的相關性則都不明顯；另外，交易量之間皆有著極高的正相關性，且隨著時間離越久而呈現遞減，交易量與各 lag 的報酬率則是都呈現相近的負相關性。

```r
corrplot(cor(SPY.return.train), method = "number")
```



0.3 Task 1 : Regression

0.3.1 2.1.a Random forest

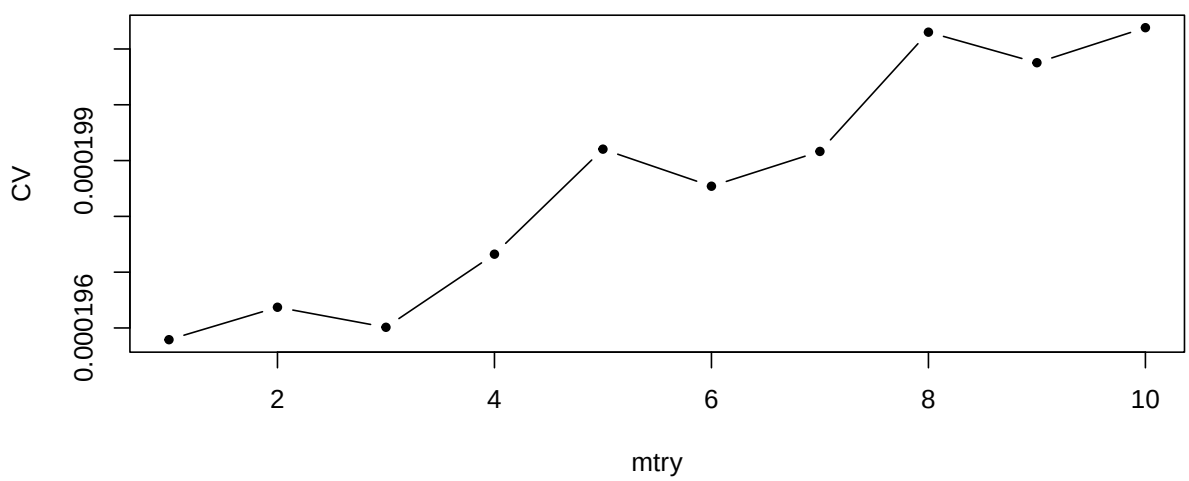
對於預測每日報酬率，我們先採用 Random forest 來進行預測。

```
rf.cvmatrix <- matrix(NA, nrow = 5, ncol = ncol(SPY.return.train[,-1]))
for (i in 1:5){
  train.cv <- SPY.return.train[cvindex[[i]],]
  valid.cv <- SPY.return.train[validindex[[i]],]
  for (j in 1:ncol(train.cv[,-1])){
    set.seed(111024523)
    rf.model <- randomForest(r ~ ., data = train.cv, mtry = j, importance = TRUE, ntree = 1000)
```

```
    rf.yhat <- predict(rf.model, newdata = valid.cv)
    rf.cvmatrix[i,j] <- mean((rf.yhat - valid.cv$r)^2)
  }
}
saveRDS(rf.cvmatrix, "C:/Users/stat-pc/Documents/111 統學/HW5/rf21a.rds")
```

先固定樹的數量為 1000 棵，CV 的結果則顯示最低為 mtry=1 時。

```
rf.cvmatrix <- readRDS("rf21a.rds")
rf.cvave <- apply(rf.cvmatrix, 2, mean)
rf.cvse <- sqrt(apply(((rf.cvmatrix - matrix(rep(rf.cvave, 5), byrow = TRUE, nrow = 5))^2)/4,
                      2, sum)/5)
plot(x = 1:length(rf.cvave), y = rf.cvave, type = "b", pch = 20, xlab = "mtry", ylab = "CV")
lines(x = 1:length(rf.cvave), y = rf.cvave+2*rf.cvse, lty = 2, col = "red")
lines(x = 1:length(rf.cvave), y = rf.cvave-2*rf.cvse, lty = 2, col = "red")
```

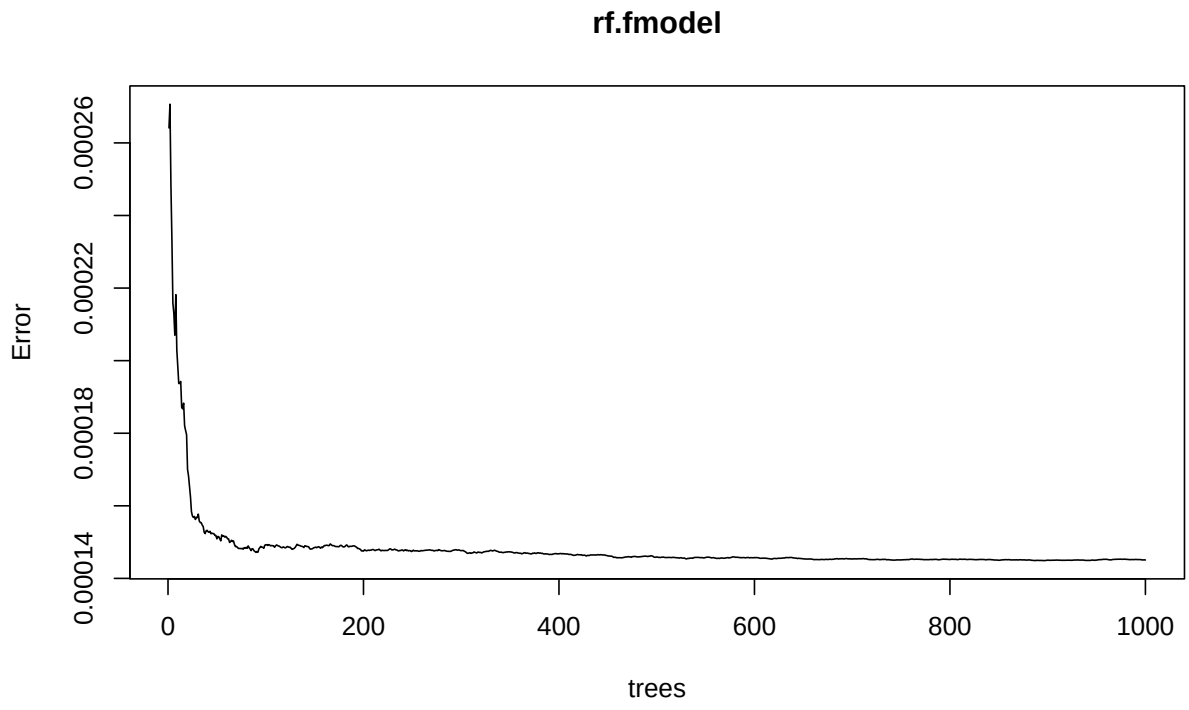



```
#legend(6.5, 950000, legend = c("CV", "CI"), lty = 1:2)
which.min(rf.cvave)
```

```
## [1] 1
```

利用 mtry=1 時再去挑選樹的數量，由圖中判斷趨於平穩時大約為 500 棵樹時。

```
set.seed(111024523)
rf.fmodel <- randomForest(r ~ ., data = SPY.return.train, mtry = 1,
                          importance = TRUE, ntree = 1000)
plot(rf.fmodel) #b=500
```

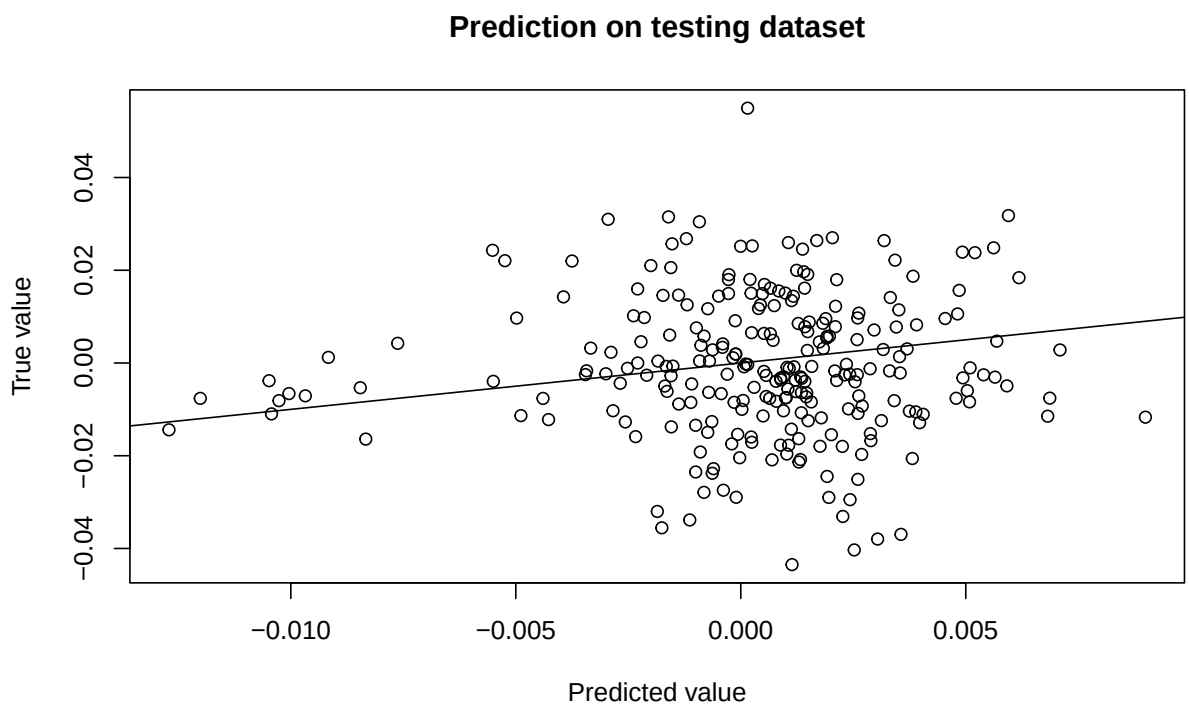


我們將 RMSE 的結果跟只利用平均數去預測的 RMSE(當作 Baseline) 比較，結果比 Baseline 高一些；畫圖呈現預測結果和實際值之散佈圖則亦顯示其預測結果並沒有到太好，幾乎都分布在預測準確線 (黑色實線) 上下。

```
set.seed(111024523)
rf.finalmodel <- randomForest(r ~ ., data = SPY.return.train, mtry = 1, ntree = 500)
rf.fyhat <- predict(rf.finalmodel, newdata = SPY.return.test)
rf.rmse <- sqrt(mean((rf.fyhat - SPY.return.test$r)^2))
rf.rmsef <-data.frame(RMSE = round(rf.rmse,5), Baseline = round(sd(SPY.return.test$r),5))
rf.rmsef
```

```
##     RMSE Baseline
## 1 0.0155  0.01523
```

```
plot(rf.fyhat, SPY.return.test$r, xlab = "Predicted value", ylab = "True value", main = "Prediction on t
abline(0, 1)
```



0.4 2.1.b.1 SVR - Radial kernel

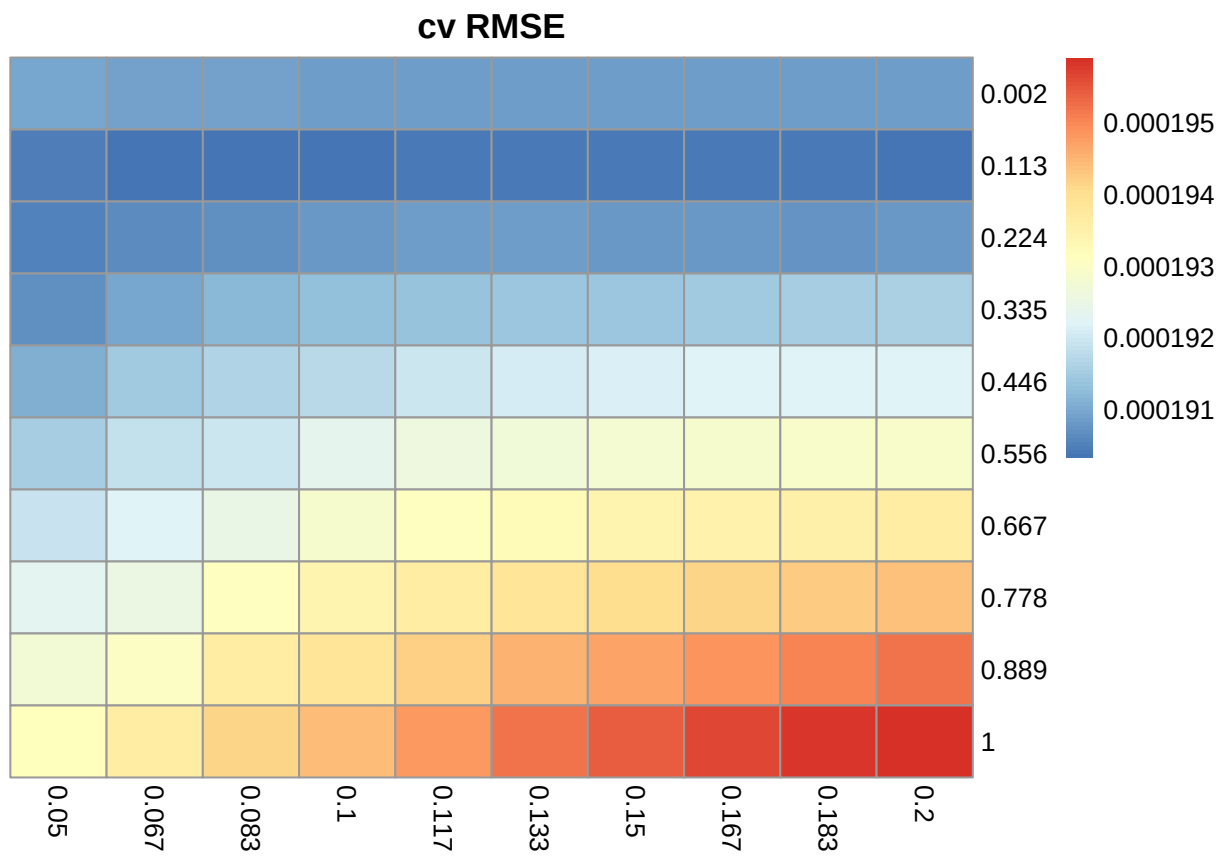
再來利用 SVM 來進行預測。我們先採用 radial kernel，並使用 CV 來挑選參數 cost 和 gamma。

```
k = 5
params1 = seq(0.002, 1, length = 10) #cost
params2 = seq(0.05, 0.2, length = 10) #gamma
cv.errors = array(NA, dim = c(k, length(params1), length(params2)))
for(i in 1:length(params1)){
  for(l in 1:length(params2)){
    print(c(params1[i], params2[l]))
    for(j in 1:k){
      set.seed(111024523)
      fit = svm(r~., data =  SPY.return.train[cvindex[[j]],], kernel = "radial",
                cost = params1[i], gamma = params2[l])
      pred = predict(fit, newdata = SPY.return.train[validindex[[j]],])
      cv.errors[j, i, l] = mean((pred - SPY.return.train[validindex[[j]],]$r)^2)
    }
  }
}

fold.n = c(rep(150, 5))

cv = apply(cv.errors*array(matrix(rep(fold.n, length(params1)), nrow = k),
                           dim = c(k, length(params1), length(params2))),
           c(2, 3), sum)/(150*5)
rownames(cv) = round(params1, 3); colnames(cv) = round(params2, 3)
opt.params = c(params1[ifelse(which.min(cv) %% 10 == 0, 10, which.min(cv) %% 10)],
```

```
                params2[ifelse(which.min(cv) %% 10 == 0, which.min(cv) %/% 10,
                                which.min(cv) %/% 10 + 1)])
set.seed(111024523)
bestsvm = svm(r~., data =  SPY.return.train, kernel = "radial",
          cost = opt.params[1], gamma = opt.params[2])
saveRDS(cv, "cv21b.rds")
saveRDS(opt.params, "opt21b.rds")
saveRDS(bestsvm, "svm21b.rds")
```

以 cost 之值 0.002 至 1、gamma 之值 0.05 至 0.2 的範圍各切 10 份來進行 CV 的挑選，由下面的 heatmap 和最小 RMSE 的參數可以得到 (cost,gamma)=(0.113,0.067) 為挑選後參數。

```
cv <- readRDS("cv21b.rds")
opt.params <- readRDS("opt21b.rds")
pheatmap(cv, cluster_rows = FALSE, cluster_cols = FALSE, main = "cv RMSE")
```




```
round(opt.params,3)
```

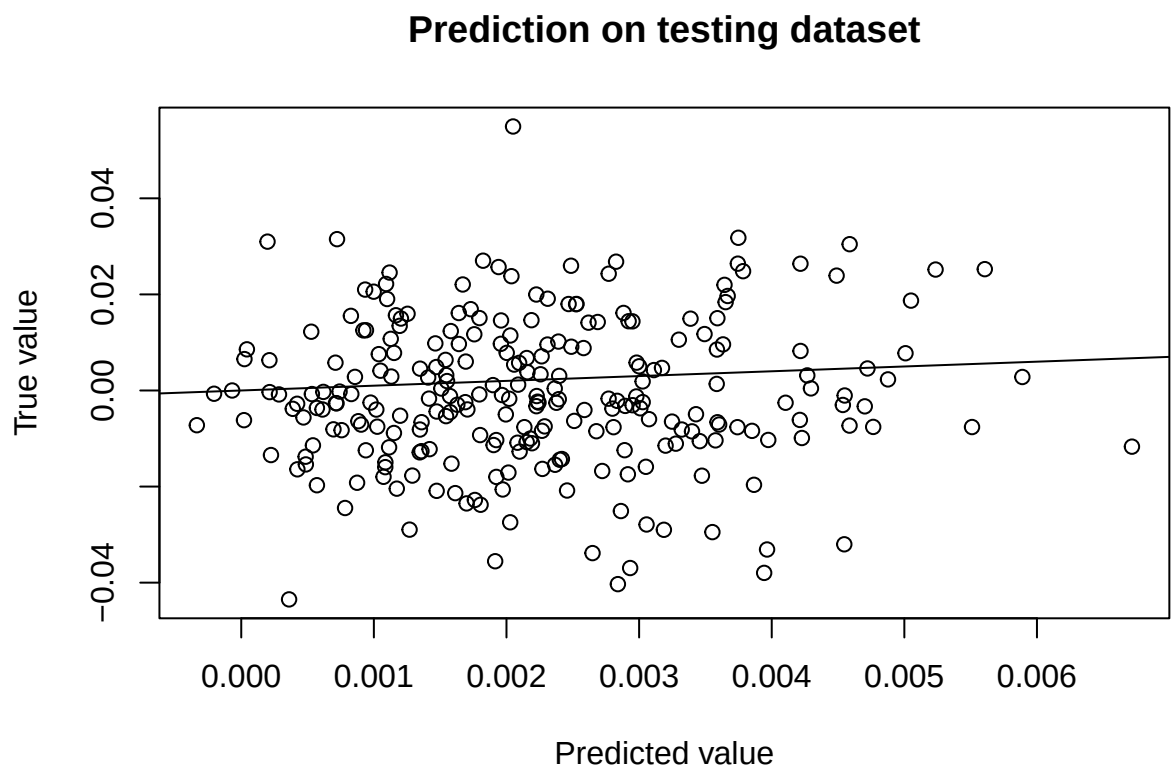
```
## [1] 0.113 0.067
```

我們將 RMSE 的結果跟只利用平均數去預測的 RMSE(當作 Baseline) 比較，結果比 Baseline 高一些；畫圖呈現預測結果和實際值之散佈圖則亦顯示其預測結果並不太好，除了幾乎都分布在預測準確線 (黑色實線) 上下外，預測值幾乎都沒有預測到負值。

```
bestsvm <- readRDS("svm21b.rds")
ypred <- predict(bestsvm, SPY.return.test)
svm.rmse <- sqrt(mean((ypred - SPY.return.test$r)^2))
svm.rmsef <-data.frame(RMSE = round(svm.rmse,5), Baseline = round(sd(SPY.return.test$r),5))
svm.rmsef
```

```
##      RMSE Baseline
## 1 0.01547  0.01523
```

```
plot(ypred, SPY.return.test$r, xlab = "Predicted value", ylab = "True value",
     main = "Prediction on testing dataset")
abline(0, 1)
```



0.5 2.1.b.2 SVR - Linear kernel

這一部分我們改成 linear kernel，並使用 CV 來挑選參數 cost。

```
k = 5
params1 = seq(0.002, 1, length = 50) #cost
cv.errors = matrix(NA, nrow = k, ncol = length(params1))
for(i in 1:length(params1)){
  print(params1[i])
  for(j in 1:k){
```

```
    set.seed(111024523)
    fit = svm(r~., data =  SPY.return.train[cvindex[[j]],], kernel = "linear", cost = params1[i])
    pred = predict(fit, newdata = SPY.return.train[validindex[[j]],])
    cv.errors[j, i] = mean((pred - SPY.return.train[validindex[[j]],]$r)^2)
  }
}

fold.n = c(rep(150, 5))

cv = as.data.frame(t(apply(cv.errors*matrix(rep(fold.n, length(params1)), nrow = k), 2, sum)/(150*5)))
colnames(cv) = round(params1, 3)
opt.param = params1[which.min(cv)]
set.seed(111024523)
bestsvm = svm(r~., data =  SPY.return.train, kernel = "linear",
          cost = opt.param)
saveRDS(opt.param, "opt21b2.rds")
saveRDS(bestsvm, "svm21b2.rds")
```

以 cost 之值 0.002 至 1 的範圍切 10 份來進行 CV 的挑選，挑選最小 RMSE 的參數為 cost=0.369。

```
opt.param <- readRDS("opt21b2.rds")
round(opt.param,3)
```

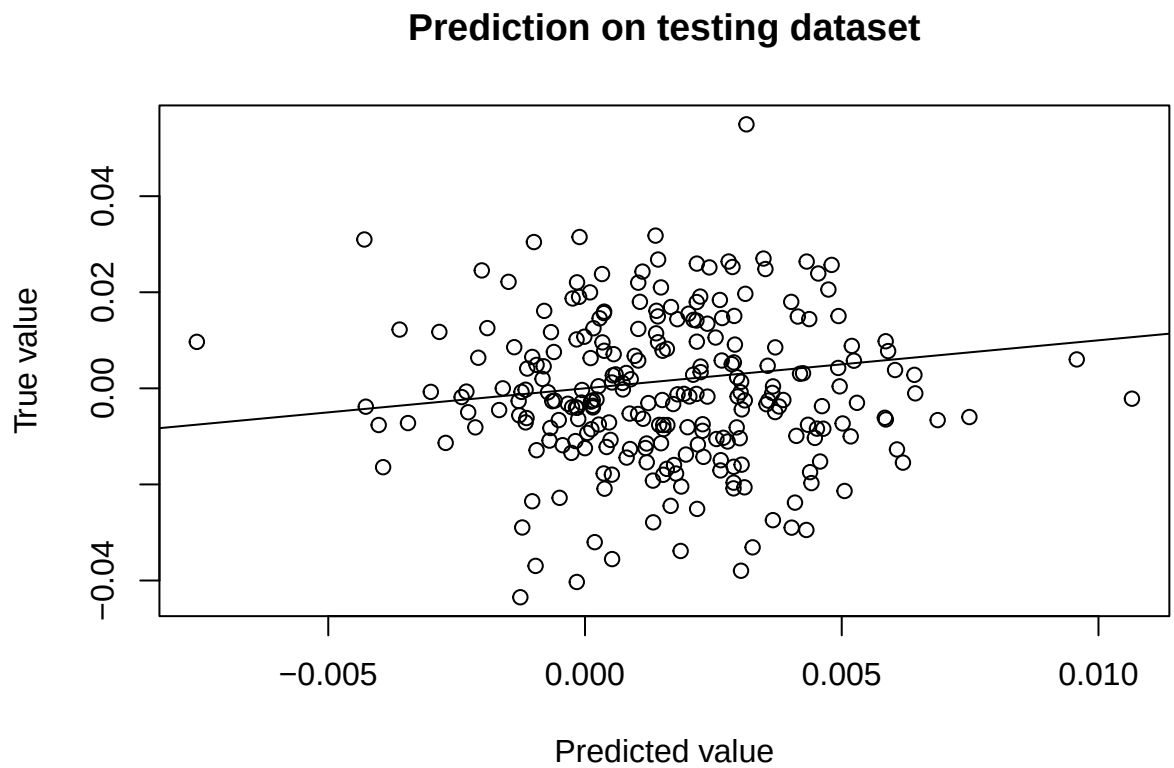
```
## [1] 0.369
```

我們將 RMSE 的結果跟只利用平均數去預測的 RMSE(當作 Baseline) 比較，結果比 Baseline 高一些；畫圖呈現預測結果和實際值之散佈圖則亦顯示其預測結果並不太好，幾乎都分布在預測準確線 (黑色實線) 上下，雖然其 RMSE 卻比 radial kernel 的還高一些，但預測值相比 radial kernel 時有預測到負值，預測的結果比較合理。

```
bestsvm <- readRDS("svm21b2.rds")
ypred <- predict(bestsvm, SPY.return.test)
svm.rmse <- sqrt(mean((ypred - SPY.return.test$r)^2))
svm.rmsef2 <-data.frame(RMSE = round(svm.rmse,5), Baseline = round(sd(SPY.return.test$r),5))
svm.rmsef2
```

```
##     RMSE Baseline
## 1 0.0156  0.01523
```

```
plot(ypred, SPY.return.test$r, xlab = "Predicted value", ylab = "True value",
     main = "Prediction on testing dataset")
abline(0, 1)
```



0.6 Task 2 : Classification

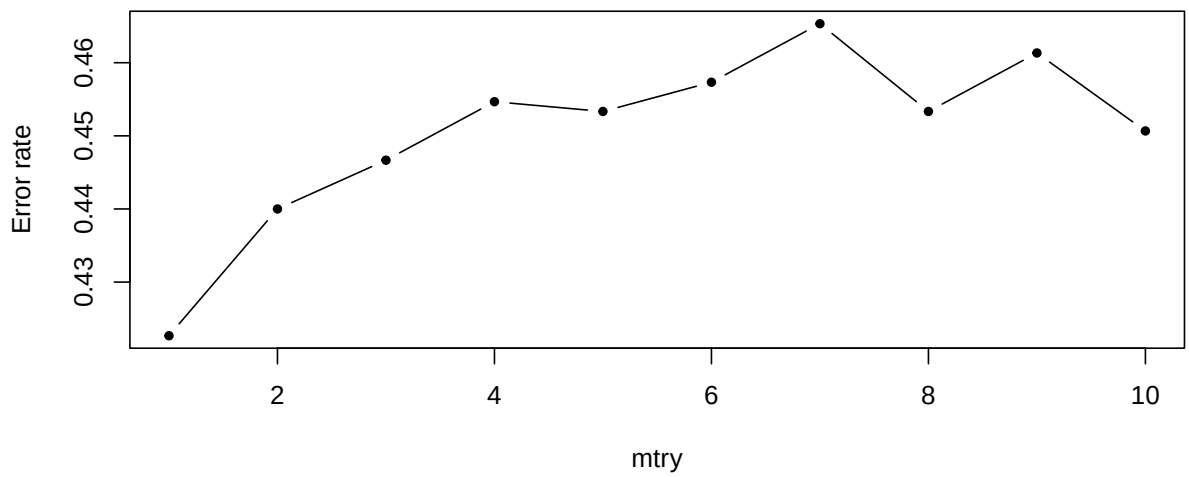
0.6.1 2.2.a Random forest

對於預測每日報酬率趨勢，我們亦先採用 Random forest 來進行預測。

```
rf.errmatrix2 <- matrix(NA, nrow = 5, ncol = ncol(SPY.trend.train[,-1]))
for (i in 1:5){
  train.cv <- SPY.trend.train[cvindex[[i]],]
  valid.cv <- SPY.trend.train[validindex[[i]],]
  for (j in 1:ncol(train.cv[,-1])){
    set.seed(111024523)
    rf.model <- randomForest(Direction ~ ., data = train.cv, mtry = j,
                             importance = TRUE, ntree = 5000)
    rf.yhat <- predict(rf.model, newdata = valid.cv)
    rf.confmatrix = table(rf.yhat,valid.cv$Direction)
    rf.errmatrix2[i,j] <- 1-(sum(diag(rf.confmatrix))/sum(rf.confmatrix))
  }
}
saveRDS(rf.errmatrix2, "rf22a.rds")
```

先固定樹的數量為 5000 棵，CV 的結果則顯示最低為 mtry=1 時。

```
rf.errmatrix2 <- readRDS("rf22a.rds")
rf.errave2 <- apply(rf.errmatrix2, 2, mean)
rf.errse2 <- sqrt(apply(((rf.errmatrix2 - matrix(rep(rf.errave2, 5), byrow = TRUE, nrow = 5))^2)/4,
                        2, sum)/5)
plot(x = 1:length(rf.errave2), y = rf.errave2, type = "b", pch = 20,
     xlab = "mtry", ylab = "Error rate")
```

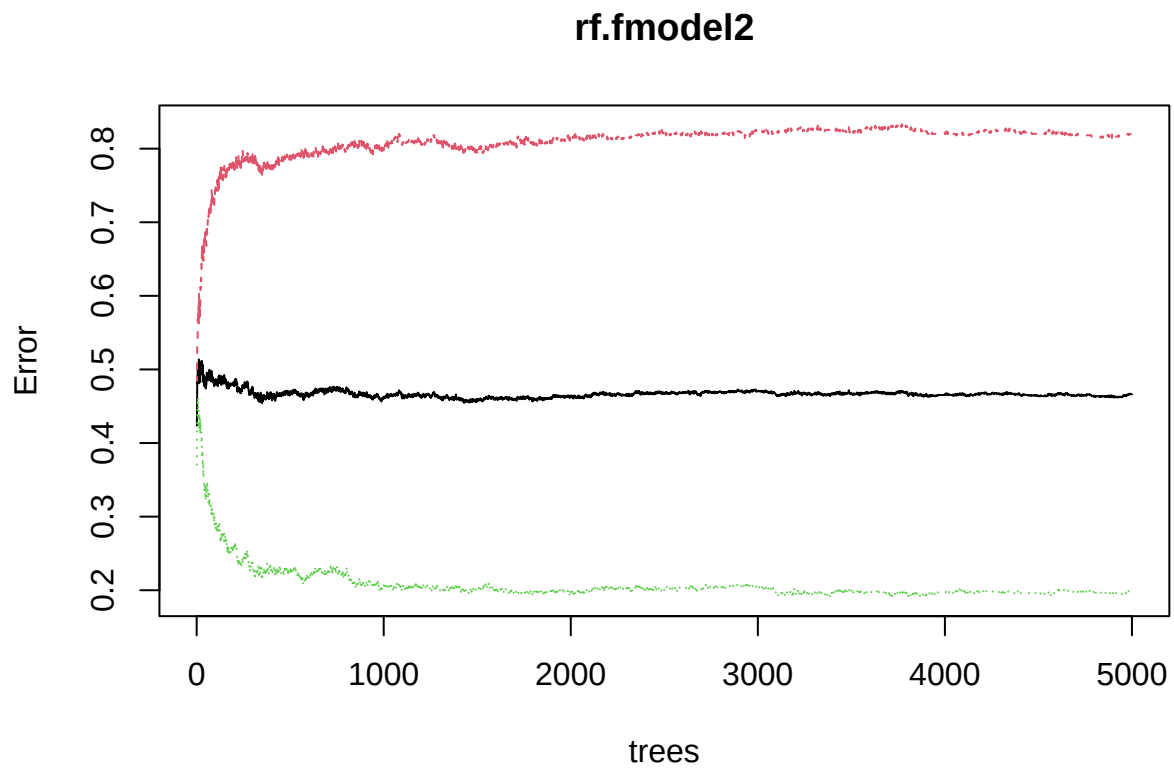



```
which.min(rf.errave2) #mtry=1
```

```
## [1] 1
```

利用 mtry=1 時再去挑選樹的數量，由圖中判斷趨於平穩時大約為 1500 棵樹時。其中圖上的 error rate 為 OOB error rate、紅線為 false negative rate、綠線為 false positive rate。

```
set.seed(111024523)
rf.fmodel2 <- randomForest(Direction ~ ., data = SPY.trend.train, mtry = 1, importance = TRUE, ntree = 5
plot(rf.fmodel2) #b=1500
```



下方的混淆矩陣和 accuracy rate 大約為 0.482，看起來表現並不是很好。

```
set.seed(111024523)
rf.finalmodel2 <- randomForest(Direction ~ ., data = SPY.trend.train, mtry = 1, importance = F, ntree =
rf.fyhat2 <- predict(rf.finalmodel2, newdata = SPY.trend.test)
rfconfmatrix2 <- table(rf.fyhat2, SPY.trend.test$Direction)
rf.ferr2 <- 1-sum(diag(rfconfmatrix2))/sum(rfconfmatrix2)
rfconfmatrix2
```

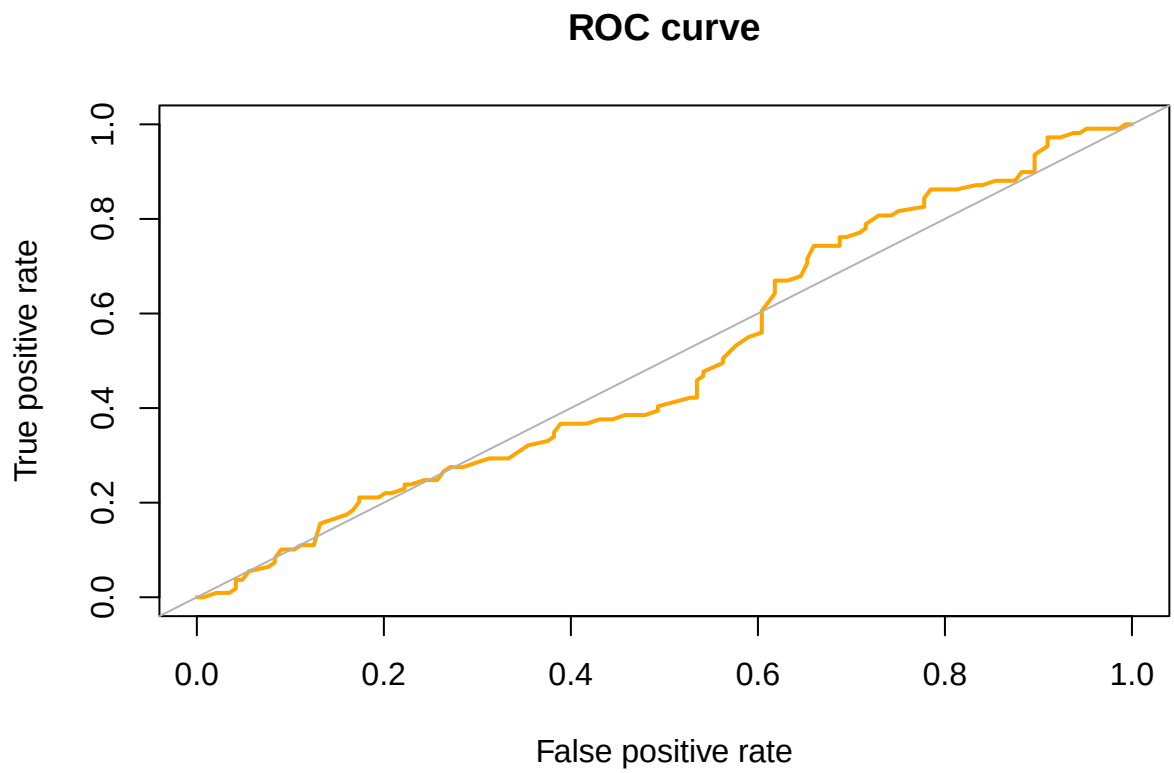
```
##
## rf.fyhat2 Down  Up
##      Down   32  19
##      Up    112  90
```

```
1-rf.ferr2
```

```
## [1] 0.4822134
```

另外，ROC curve 亦顯示其在 45 度線上下震盪，且很取決於 threshold 值的設定，AUC 亦只有 0.501，結果亦並不是很好。

```
rf.fprob2 <- predict(rf.finalmodel2, newdata = SPY.trend.test, type = "prob")
roc.curve(SPY.trend.test$Direction, rf.fprob2[,2], plotit = T, col = "orange")
```




```
## Area under the curve (AUC): 0.501
```

0.7 2.2.b.1 SVM - Radial kernel

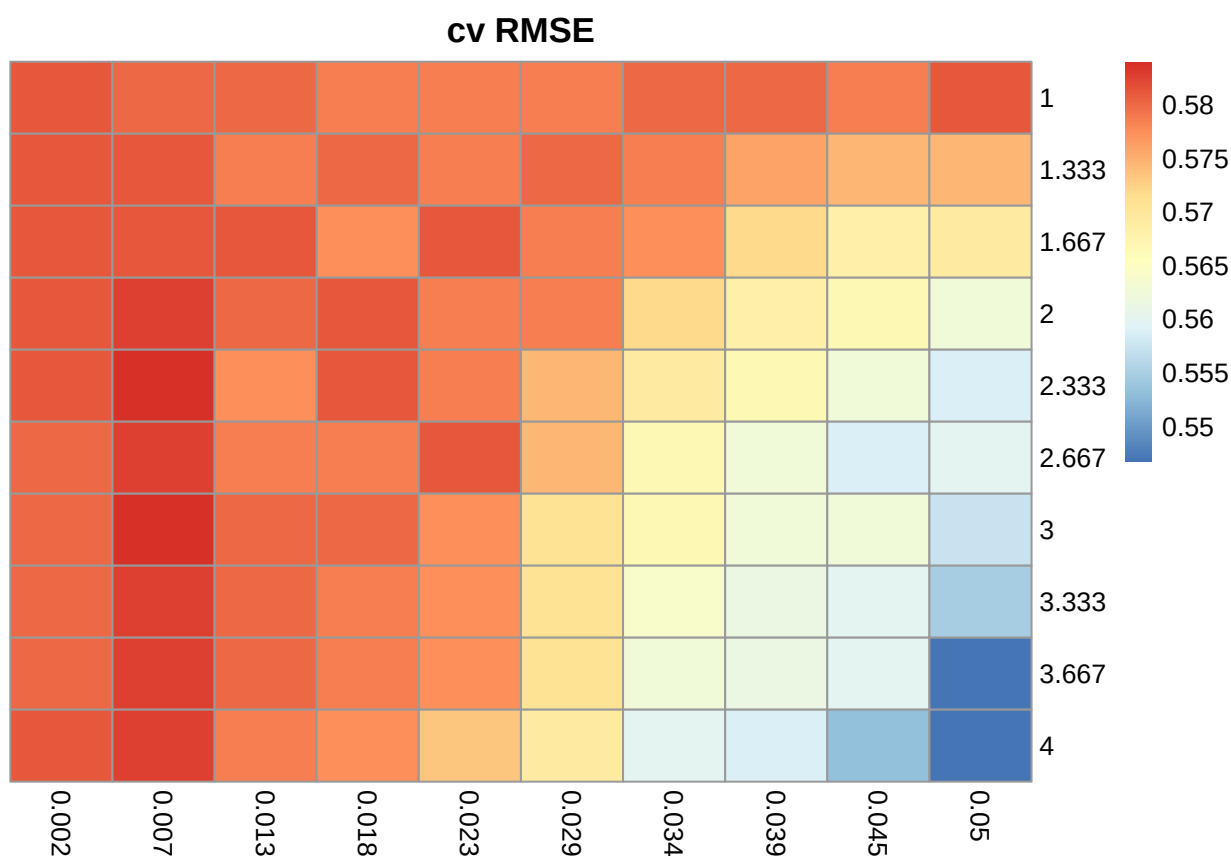
再來利用 SVM 來進行預測。我們採用 radial kernel，並使用 CV 來挑選參數 cost 和 gamma。

```
k = 5
params1 = seq(1, 4, length = 10) #cost
params2 = seq(0.002, 0.05, length = 10) #gamma
cv.errors = array(NA, dim = c(k, length(params1), length(params2)))
for(i in 1:length(params1)){
  for(l in 1:length(params2)){
    print(c(params1[i], params2[l]))
    for(j in 1:k){
      set.seed(111024523)
      fit = svm(Direction~., data =  SPY.trend.train[cvindex[[j]],], kernel = "radial",
                cost = params1[i], gamma = params2[l])
      pred = predict(fit, newdata = SPY.trend.train[validindex[[j]],])
      confuse = table(SPY.trend.train[validindex[[j]],]$Direction, pred)
      acc = sum(diag(confuse))/sum(confuse)
      cv.errors[j, i, l] = acc
    }
  }
}
fold.n = c(rep(150, 5))
```

```
cv2 = apply(cv.errors*array(matrix(rep(fold.n, length(params1)), nrow = k),
                            dim = c(k, length(params1), length(params2))),
           c(2, 3), sum)/(150*5)
rownames(cv2) = round(params1, 3); colnames(cv2) = round(params2, 3)
pheatmap(cv2, cluster_rows = FALSE, cluster_cols = FALSE, main = "cv Accuracy")
opt.params2 = c(params1[ifelse(which.max(cv2) %% 10 == 0, 10, which.max(cv2) %% 10)],
               params2[ifelse(which.max(cv2) %% 10 == 0, which.max(cv2) %/% 10,
                              which.max(cv2) %/% 10 + 1)])

set.seed(111024523)
bestsvm2 = svm(Direction~., data =  SPY.trend.train, kernel = "radial",
          cost = opt.params2[1], gamma = opt.params2[2], probability = T)
saveRDS(cv2, "cv22b.rds")
saveRDS(opt.params2, "opt22b.rds")
saveRDS(bestsvm2, "svm22b.rds")
```

以 cost 之值 1 至 4、gamma 之值 0.002 至 0.05 的範圍各切 10 份來進行 CV 的挑選，由下面的 heatmap 和最小 RMSE 的參數可以得到 (cost,gamma)=(2.333,0.007) 為挑選後參數。

```
cv2 <- readRDS("cv22b.rds")
opt.params2 <- readRDS("opt22b.rds")
pheatmap(cv2, cluster_rows = FALSE, cluster_cols = FALSE, main = "cv RMSE")
```



```
opt.params2
```

```
## [1] 2.333333333 0.007333333
```

下方的混淆矩陣和 accuracy rate 大約為 0.431，且預測值皆預測趨勢往上，看起來表現非常不好。

```
bestsvm2 <- readRDS("svm22b.rds")
set.seed(111024523)
ypred2 <- predict(bestsvm2, SPY.trend.test, probability = T)
svmconfmatrix2 <- table(ypred2, SPY.trend.test$Direction)
svmconfmatrix2
```

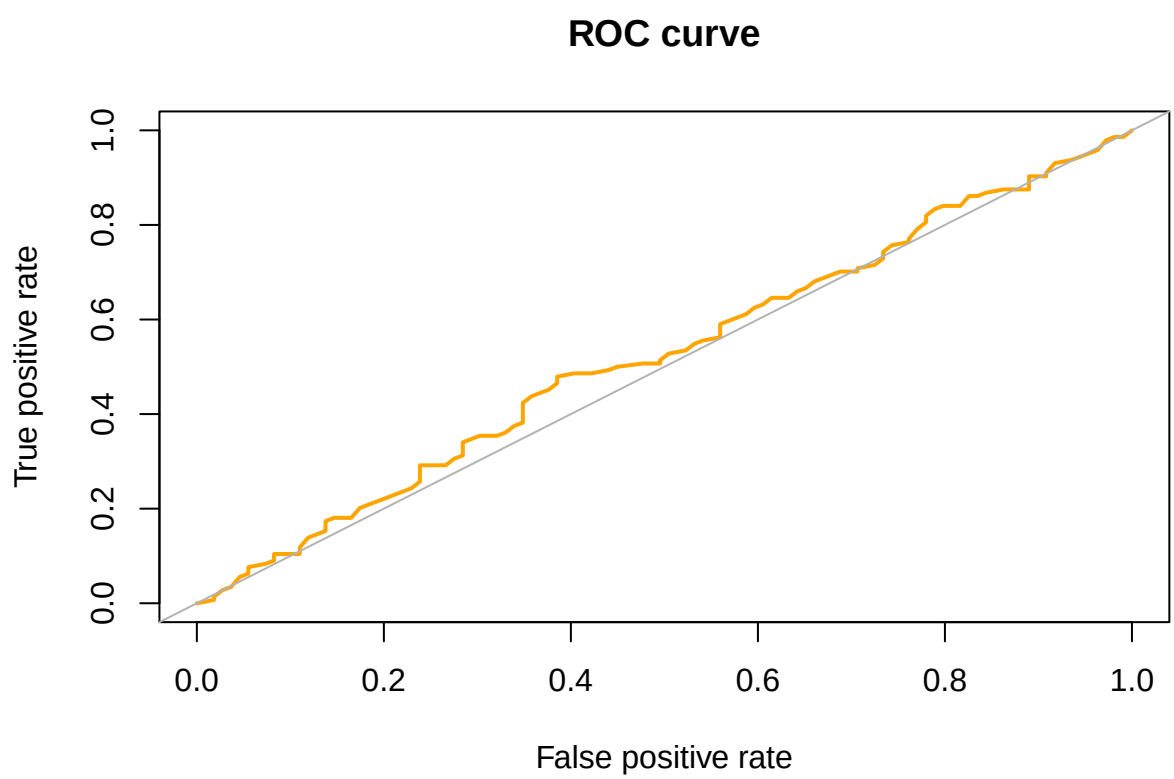
```
##
## ypred2 Down  Up
##   Down    0   0
##   Up    144 109
```

```
svmferr2 <- 1-(sum(diag(svmconfmatrix2))/sum(svmconfmatrix2))
1-svmferr2
```

```
## [1] 0.43083
```

雖然 ROC curve 幾乎在 45 度線上方，AUC 有 0.523，但鑑於上方混淆矩陣的結果，我們決定對於機率值進行觀察。

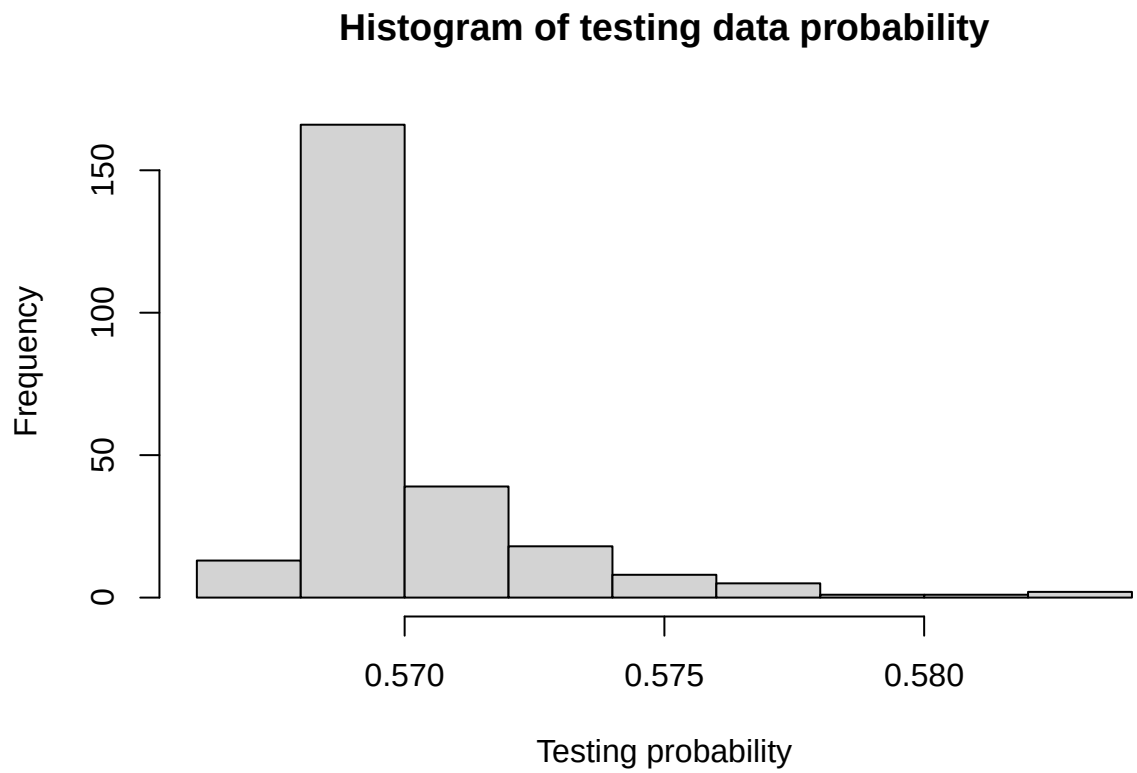
```
roc.curve(SPY.trend.test$Direction, attr(ypred2, "probabilities")[,1], plotit = T, col = "orange")
```



```
## Area under the curve (AUC): 0.523
```

由下方直方圖可以看到，testing data 的預測機率值都非常接近，因此 threshold 的設定要設定地很細才會對分類有影響，而預設 threshold 為 0.5，因此混淆矩陣才會一致分至 Up。

```
hist(attr(ypred2, "probabilities")[,1], main = "Histogram of testing data probability", xlab = "Testing
```



0.8 2.2.b.2 SVM - Linear kernel

再來用 linear kernel，並使用 CV 來挑選參數 cost。

```
k = 5
params1 = seq(10, 50, length = 50) #cost
cv.errors = matrix(NA, nrow = k, ncol = length(params1))
for(i in 1:length(params1)){
  print(params1[i])
  for(j in 1:k){
    set.seed(111024523)
    fit = svm(Direction~., data =  SPY.trend.train[cvindex[[j]],], kernel = "linear", cost = params1[i]
    pred = predict(fit, newdata = SPY.trend.train[validindex[[j]],])
    confuse = table(SPY.trend.train[validindex[[j]],]$Direction, pred)
    acc = sum(diag(confuse))/sum(confuse)
    cv.errors[j, i] = acc
  }
```

```
}
fold.n = c(rep(150, 5))
cv2 = as.data.frame(t(apply(cv.errors*matrix(rep(fold.n, length(params1)), nrow = k), 2, sum)/(150*5)))
colnames(cv2) = round(params1, 3)
opt.param2 = params1[which.max(cv2)]

set.seed(111024523)
bestsvm2 = svm(Direction~., data =  SPY.trend.train, kernel = "linear",
          cost = opt.param2, probability = T)
saveRDS(cv2, "cv22b2.rds")
saveRDS(opt.param2, "opt22b2.rds")
saveRDS(bestsvm2, "svm22b2.rds")
```

試了很多種 cost 的 sequence 去做 cv，但發現 cost 之值不管為多少其正確率都是一樣 0.5813，所以最後以 cost=10 為挑選後參數。

```
cv2 <- readRDS("cv22b2.rds")
opt.param2 <- readRDS("opt22b2.rds")
summary(t(cv2))
```

```
##        V1
##  Min.   :0.5813
##  1st Qu.:0.5813
##  Median :0.5813
##  Mean   :0.5813
##  3rd Qu.:0.5813
##  Max.   :0.5813
```

```
opt.param2
```

```
## [1] 10
```

下方的混淆矩陣和 accuracy rate 大約為 0.431，且預測值皆預測趨勢往上，看起來表現與 radial kernel 時一樣非常不好。

```
bestsvm2 <- readRDS("svm22b2.rds")
set.seed(111024523)
ypred2 <- predict(bestsvm2, SPY.trend.test, probability = T)
svmconfmatrix22 <- table(ypred2, SPY.trend.test$Direction)
svmconfmatrix22
```

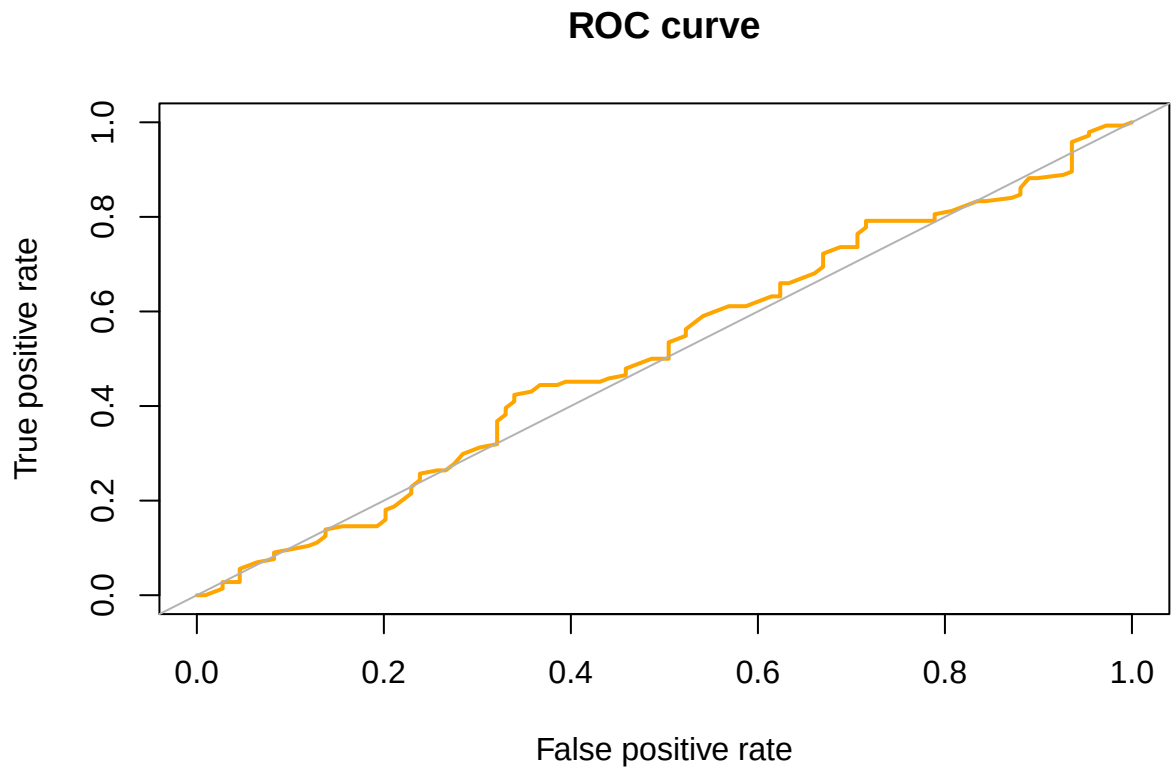
```
##
## ypred2 Down  Up
##   Down    0   0
##   Up    144 109
```

```
svmferr22 <- 1-(sum(diag(svmconfmatrix2))/sum(svmconfmatrix2))
1-svmferr22
```

```
## [1] 0.43083
```

雖然 ROC curve 幾乎在 45 度線上方，AUC 有 0.514，但鑑於上方混淆矩陣的結果，我們決定對於機率值進行觀察。

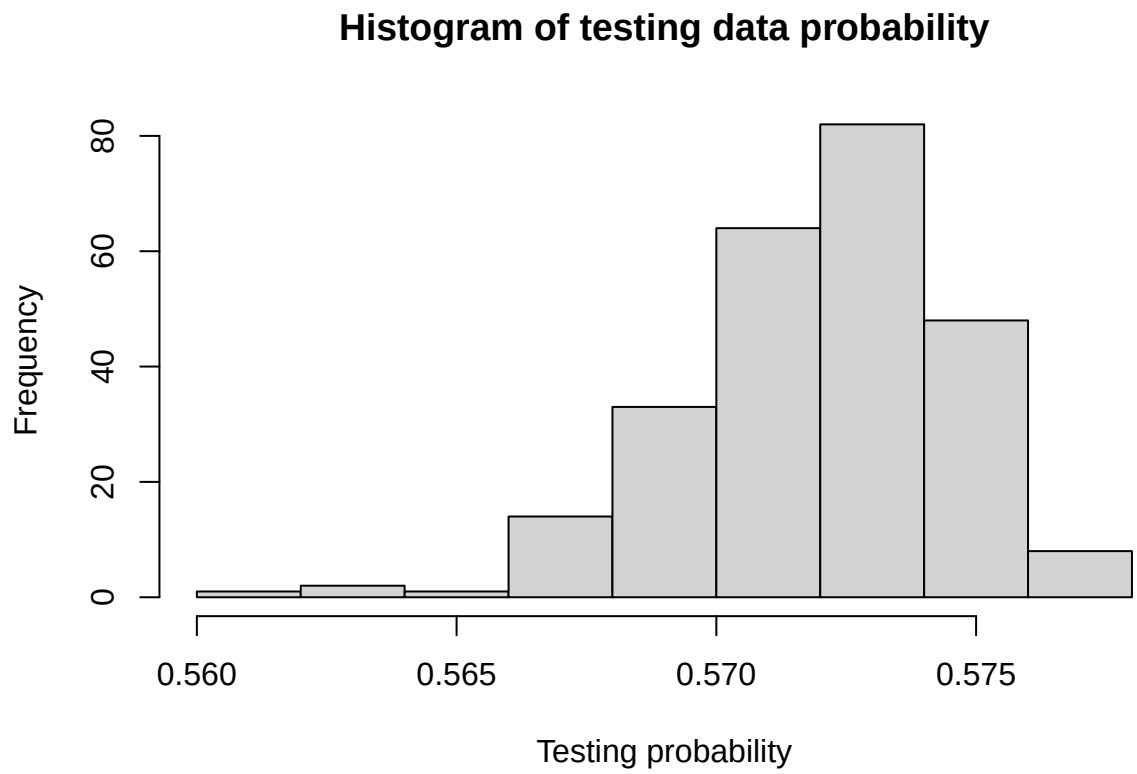
```
roc.curve(SPY.trend.test$Direction, attr(ypred2, "probabilities")[,1], plotit = T, col = "orange")
```




```
## Area under the curve (AUC): 0.514
```

由下方直方圖可以看到，混淆矩陣的原因與 radial kernel 時相同。

```
hist(attr(ypred2, "probabilities")[,1], main = "Histogram of testing data probability", xlab = "Testing
```



1 2.2.c Logistics regression

最後再用羅吉斯迴歸試試預測結果，先用 AIC 進行選模。

```
logi_model <- glm(Direction ~., data = SPY.trend.train, family = binomial)
aic_logi <- step(logi_model, direction = "both", k = 2)
saveRDS(aic_logi, "logi22c.rds")
```

AIC 最後篩出以下三個變數，分別為 r.Lag.1、v.Lag.1、v.Lag.5，利用這些變數進行最後建模。

```
aic_logi <- readRDS("logi22c.rds")
aic_logi$coefficients
```

```
## (Intercept)     r.Lag.1     v.Lag.1     v.Lag.5
##    0.369614  -15.780450   -3.919186    2.994535
```

```
logi_fmodel <- glm(Direction ~ r.Lag.1 + v.Lag.1 + v.Lag.5, data = SPY.trend.train, family = binomial)
logi_prob <- predict(logi_fmodel, SPY.trend.test, type = "response")
logi_pred <- rep("Down", nrow(SPY.trend.test))
logi_pred[logi_prob > .5] <- "Up"
logi_testcase <- SPY.trend.test$Direction
logi_cm <- table(logi_pred, logi_testcase)
logi_facc <- sum(diag(logi_cm))/sum(logi_cm)
```

利用這些變數進行建模後的混淆矩陣如下，看起來也是大多都預測趨勢往上，經計算正確率得到值為約 0.451，表現沒有很好。

```
logi_cm
```

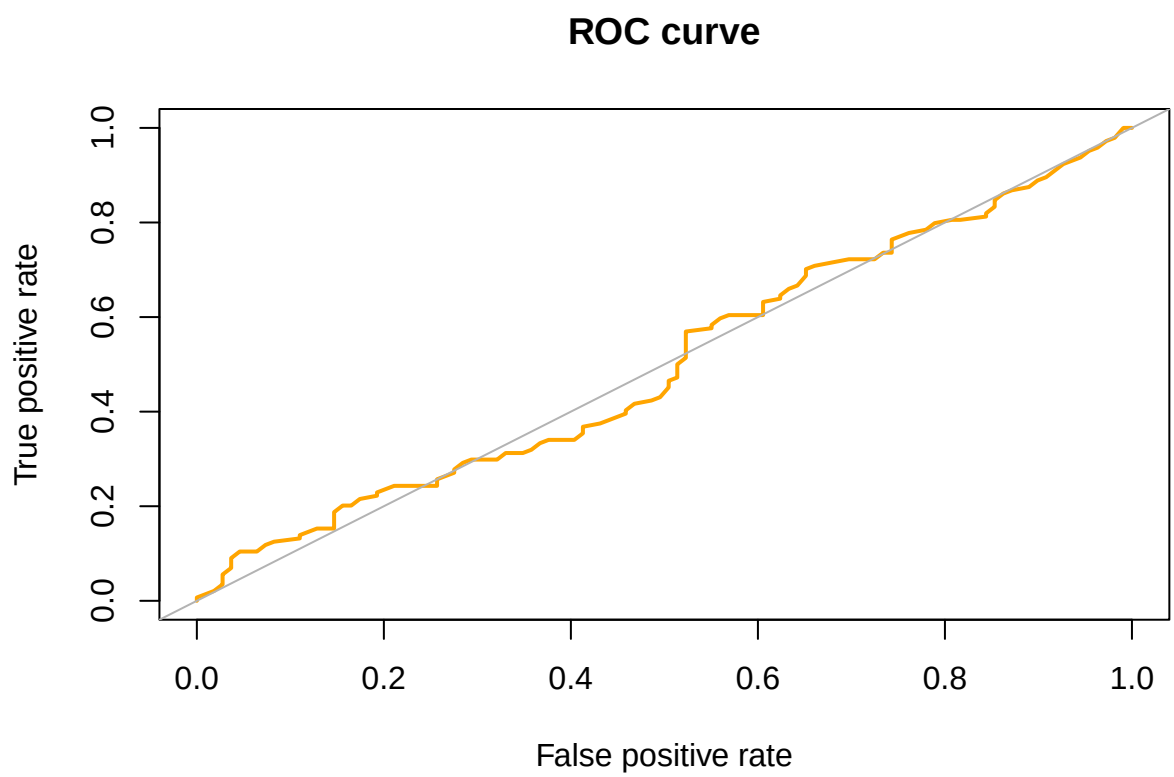
```
##          logi_testcase
## logi_pred Down  Up
##      Down   17  12
##      Up    127  97
```

```
logi_facc
```

```
## [1] 0.4505929
```

ROC curve 顯示其在 45 度線上下震盪，且很取決於 threshold 值的設定，AUC 亦只有 0.502，結果並不是很好。

```
roc.curve(SPY.trend.test$Direction, logi_prob, plotit = T, col = "orange")
```




```
## Area under the curve (AUC): 0.502
```

再來試用 BIC 進行選模。

```
bic_logi <- step(logi_model, direction = "both", k = log(nrow(SPY.trend.train)))
saveRDS(bic_logi, "logi22cbic.rds")
```

選模結果顯示將所有變數都篩掉，因此羅吉斯迴歸的結果便是前面 AIC 的選模結果。

```
bic_logi <- readRDS("logi22cbic.rds")
bic_logi$coefficients
```

```
## (Intercept)
##   0.2811674
```

1.1 2.3 Summary

最後來總結一下對於 daily return 的預測、報酬率趨勢之各模型預測結果。

```
second.sum.reg <- data.frame(randomforest = rf.rmsef[,1],
                        svm.radial = svm.rmsef[,1],
                        svm.linear = svm.rmsef2[,1],
                        baseline = rf.rmsef[,2])
second.sum.class <- data.frame(randomforest = 1-rf.ferr2,
                        svm.radial = 1-svmferr2,
                        svm.linear = 1-svmferr22,
                        logistics = logi_facc,
                        baseline = 0.5)
```

在 daily return 中，RMSE 由高至低分別是 radial 當 kernel 的 SVM、random forest、linear 當 kernel 的 SVM，但這三個方法皆不比只利用平均數去預測的結果還好，代表其實預測的比擲銅板隨機預測的結果還差。

```
rownames(second.sum.reg) <- "Testing RMSE"
second.sum.reg
```

```
##              randomforest svm.radial svm.linear baseline
## Testing RMSE       0.0155    0.01547     0.0156  0.01523
```

在分類報酬率趨勢的部分，Accuracy 由高至低分別是 random forest、logistics regression、radial 當 kernel 的 SVM 和 linear 當 kernel 的 SVM 則有一樣的值，但這三個方法的正確率皆低於 0.5，代表其實預測的比擲銅板隨機預測的結果還差。

```
rownames(second.sum.class) <- "Testing Error rate"
second.sum.class
```

```
##                    randomforest svm.radial svm.linear logistics baseline
## Testing Error rate    0.4822134    0.43083    0.43083 0.4505929      0.5
```

會出現上面如此差的預測結果，我們認為很大一部份是股價 (換算成報酬率) 本身便是十分難預測走向的事物，其受到各項總經指標、政治因素、產業趨勢、投資人心理、各大法人和基金公司的操作動向等等因素的影響，這些因素的數量本身就已經很多，所以其反映於股價上的變異便更複雜、更難去找出其規律的部分，或甚至其實本質上根本沒有規律可言，造成股價呈現如此的 non-stasionary，畢竟若是真的有人能夠厲害到找到股價真實的規律和其巨大變異的變動情形，那他直接就成為巴菲特了，不過實際上沒人能做到，甚至是擁有許多財金和總經專業知識的財金人士也做不到，因此如此低的預測結果我們欣然接受。

不過，我們仍想試試是否能將正確率提高到至少 0.5，考慮到股價資料為一時間序列資料，若是以後我們有學習到關於時序方面的知識與各種適合的時序模型，比如說 AR 或 ARIMA model，或許能夠對於此資料做更適當的處理，進而提升正確率也說不定。